# Projector-Generated Special Functions IV

Squarefree Depth–Time Kernels, Universal Gamma Dephasing, Möbius Resolvents, and Signed Padé Packet Models

Komorebi P.

**Abstract**

This paper develops the first joint scale-time special-function system generated by the projector-resolved calculus. The signed seed is the Möbius transfer

$$C_{a,\beta,\chi}(k,t) = \sum_{n\geq 1} \frac{\mu(n)\chi(n)}{n^{a+\mathrm{i}t}} \left(1 - \frac{1}{n^{\beta}}\right)^{k}.$$

Its long-time cross-correlation removes the arithmetic signs on the diagonal and produces a positive depth-time kernel. The kernel is proved to be positive definite and to admit an exact finite-difference coordinate system built from squarefree Dirichlet series. Its large-depth behavior is universal. After one carrier phase is removed, the equal-depth correlation converges to

$$\frac{\Gamma(\alpha + \mathrm{i}\tau/\beta)}{\Gamma(\alpha)}, \qquad \alpha = \frac{2a-1}{\beta}.$$

For the principal parameters $a = \beta = 2$, the squared dephasing envelope has the elementary closed form

$$\frac{1+\tau^2}{\cosh(\pi\tau/2)}.$$

The paper also proves a universal overlap law between different packet depths. If $k/\ell$ tends to a positive ratio, then the normalized overlap tends to

$$\operatorname{sech}^{\alpha}\left(\frac{1}{2}\log\frac{k}{\ell}\right).$$

The tilted logarithmic frequencies have explicit limiting cumulants governed by the polygamma functions. Cross-kernels between distinct Dirichlet-character sectors decay at the unconditional rate $m^{-1/(2\beta)}$ after normalization.

The signed side is encoded by a Möbius resolvent. It satisfies an exact parameter-shift equation and a Borel–Riesz bridge. Whenever a finite Hankel determinant is nonzero, a unique Padé packet model preserves the first $2m$ signed moments after $m$ packets. Unlike the positive Jacobi models of the preceding paper, signed packet nodes need not remain in the physical interval. This failure is exhibited explicitly.

The positive kernel is the long-time autocorrelation of the signed process. The Riemann hypothesis remains equivalent to the statement that the distinguished recurrence time $t = 0$ is not polynomially more coherent than the average packet power. This paper proves the complete unconditional depth-time and finite-reduction framework. It does not prove the remaining Möbius coherence bound.

All figures were generated by the accompanying Python program. The release contains 28 vector figures, matching PNG files, exact tables, 18 built-in checks, 18 independent tests, and the complete Python source.

# Status and claim boundary

The paper proves the depth-time Gram representation, the exact time-average identity, the squarefree $L$-ratio coordinate formula, the uniform Gamma dephasing limit on compact time intervals, the principal closed-form envelope, the log-frequency cumulant limit, the cross-depth overlap law, and the character cross-kernel decay theorem. It also proves the analytic signed-resolvent representation, its parameter-shift identity, the Borel–Riesz bridge, and the finite Padé packet theorem under a nonvanishing Hankel determinant.

The positive kernel theory is unconditional. The finite signed reduction is conditional only on the explicitly stated nonbreakdown determinant. None of these results proves RH. The remaining assertion is the subpolynomial growth of the principal time-zero Möbius coherence. Historical priority and full references are reserved for the consolidated end-of-series bibliography.

The logarithmic quantum interpretation is compatible with the external 2026 construction in which the number basis carries the Hamiltonian $H\,|n\rangle = \log n\,|n\rangle$ and the phase $n^{-\mathrm{i}t}$. The present work uses that coordinate but studies an autocorrelation and scale filter that are distinct from the direct zeta observable.

# Contents


Status and claim boundary i

I The signed transfer and its positive Gram lift 1

1 One signed seed and two exact evolutions 1

2 The depth–time kernel 2

3 Squarefree Dirichlet coordinates 4

II Universal depth-time asymptotics 5

4 A complex Beta asymptotic 6

5 The Gamma dephasing function 7

6 Log-frequency cumulants 11

7 The cross-depth overlap law 13

III Character packets and arithmetic decoherence 15

8 Cross-character depth-time kernels 15

IV The signed Möbius function family 17

9 The Möbius resolvent 17

10 The Borel–Riesz bridge 21

11 Signed Padé packet models 22

**V Quantum and recurrence interpretation** **25**

**12 The number-basis operator system** **25**

**13 The recurrence process and the RH boundary** **26**

**VI Worked algorithms and verification** **28**

**14 Computational route** **28**

**15 Verification ledger** **29**

**A Formula registry** **30**

**B Implementation map** **31**

**C Complete Python source** **32**

C.1 Reproduction . . . . . 48

C.2 Source boundary . . . . . 48

# List of Figures


1.1 The mathematical pipeline. The signed transfer is lifted by time averaging into a positive kernel. The positive kernel has a universal Gamma limit. The signed resolvent and its finite Padé reductions preserve the coherent information that the positive average removes. 2

2.1 The exact feature map. Depth controls the active scale. Logarithmic time rotates each number-basis packet at frequency $\log n$. Their Gram contraction is the depth-time kernel. 3

2.2 A finite depth-time Gram matrix. The matrix is Hermitian positive semidefinite. The plotted eigenvalues provide a numerical certificate for the displayed packet set. . . . . 4

3.1 Two exact coordinates for one kernel. The direct squarefree sum and the finite difference of zeta ratios agree to floating-point roundoff in the worked calculation. . . . . 5

3.2 Finite-time extraction of the diagonal kernel. Increasing the observation window suppresses off-diagonal logarithmic frequencies and approaches the exact positive packet correlation. 5

5.1 The limiting dephasing trajectory. The Gamma ratio traces a zero-free curve in the complex plane. The marked point is the normalized value at zero lag. . . . . 8

5.2 Convergence to the Gamma ratio. The dephased discrete correlations approach the same real and imaginary limit as the packet depth grows. . . . . 9

5.3 The principal closed-form dephasing envelope. The Gamma-ratio modulus and the elementary hyperbolic-cosine expression coincide throughout the displayed range. . . . . 10

5.4 The phase and its derivative. The digamma function supplies the exact group-delay law of the limiting packet correlation. . . . . 11

6.1 The centered logarithmic spectrum. The finite squarefree packet histogram approaches the log-Gamma density predicted by the dephasing function. . . . . 12

6.2 Convergence of the first two cumulants. The mean and variance settle toward explicit digamma and trigamma constants. . . . . 13

7.1 The exact limiting overlap profile. Packets at comparable depths retain substantial overlap. Packets separated multiplicatively in depth become asymptotically orthogonal. . . . . 14

7.2 Finite-depth convergence. Several absolute scales approach the same ratio-dependent law. 14

7.3 The geometry on logarithmic depth. The overlap depends asymptotically only on the difference of log depths, which produces a translation-invariant ridge in scale coordinates. 15

8.1 Modulo-five character Gram matrices. The diagonal remains normalized to one while the off-diagonal sectors weaken with increasing depth. . . . . 16

8.2 The largest off-diagonal character correlation. The finite values decrease in the direction predicted by the unconditional quarter-power law. . . . . 17

9.1 The signed transfer in logarithmic time. The signal is quasiperiodic because every integer packet rotates at its own frequency $\log n$. Increasing the depth changes the active scale but not the frequency law. . . . . 18

9.2 The unit-average-power process. The positive packet energy normalizes the long-time mean power to one. The value at $t = 0$ remains a distinguished coherent recurrence. . . . . 19

9.3 Finite time-zero coherence data. The plot is a finite computational record only. It is not used to infer the asymptotic RH bound. . . . . 19

9.4 The signed resolvent. The function is analytic on the displayed side of the physical cut. Unlike the positive resolvent, it may have zeros and sign cancellations. . . . . 20

9.5 The transfer-equation residual. Three complex evaluation points verify the parameter-shift identity to the truncation accuracy of the number-basis sum. . . . . 21

10.1 The Borel–Riesz identity. The discrete depth sequence and its continuous companion are two exact coordinates of the same Möbius transfer. . . . . 22

11.1 Positive and signed packet nodes. The positive companion reduction stays in the declared spectral interval. The signed Möbius reduction can leave it because cancellation replaces positivity. . . . . 23

11.2 Moment-exact finite reduction. Four packets preserve the first eight positive or signed moments. The later moments are predictions rather than part of the exact contract. . . . 24

11.3 Positive and signed Hankel determinants. The positive determinants remain positive. The signed determinants change sign and become increasingly ill-conditioned, which exposes the nonbreakdown boundary directly. . . . . 24

12.1 The operator realization. The signed amplitude is a coherent readout. Time averaging produces the positive depth-time kernel and removes only the off-diagonal logarithmic frequencies. . . . . 26

13.1 The exact boundary. The positive depth-time theory is unconditional. Finite signed models are exact under a visible nonbreakdown condition. The principal time-zero coherence remains open. . . . . 27

13.2 The generated function family. The paper adds one positive reproducing kernel, one zero-free Gamma dephasing limit, one signed resolvent, one log-frequency law, one character cross-kernel family, and finite signed Padé packet models. . . . . 28

# Part I

# The signed transfer and its positive Gram lift

## Chapter 1

## One signed seed and two exact evolutions

Fix

$$a > 1, \qquad \beta > 0,$$

and let $\chi$ be a Dirichlet character. Set

$$y_n = 1 - \frac{1}{n^\beta}.$$

The signed transfer is

$$\boxed{C_{a,\beta,\chi}(k,t) = \sum_{n\geq 1} \mu(n)\chi(n)n^{-a-\mathrm{i}t}y_n^k.}$$

Absolute convergence follows from $a > 1$. The variable $k$ is a discrete Euclidean depth. The variable $t$ is logarithmic time.

The shift in $k$ gives the exact recursion

$$\boxed{C_{a,\beta,\chi}(k+1,t) = C_{a,\beta,\chi}(k,t) - C_{a+\beta,\beta,\chi}(k,t).}$$

Indeed,

$$y_n^{k+1} = y_n^k - n^{-\beta}y_n^k.$$

Expanding the depth filter produces the finite-difference form

$$\boxed{C_{a,\beta,\chi}(k,t) = \sum_{j=0}^{k}(-1)^j\binom{k}{j}\frac{1}{L(a+\beta j+\mathrm{i}t,\chi)}.}$$

The Möbius packet is therefore a weighted finite-difference orbit of reciprocal $L$-values.

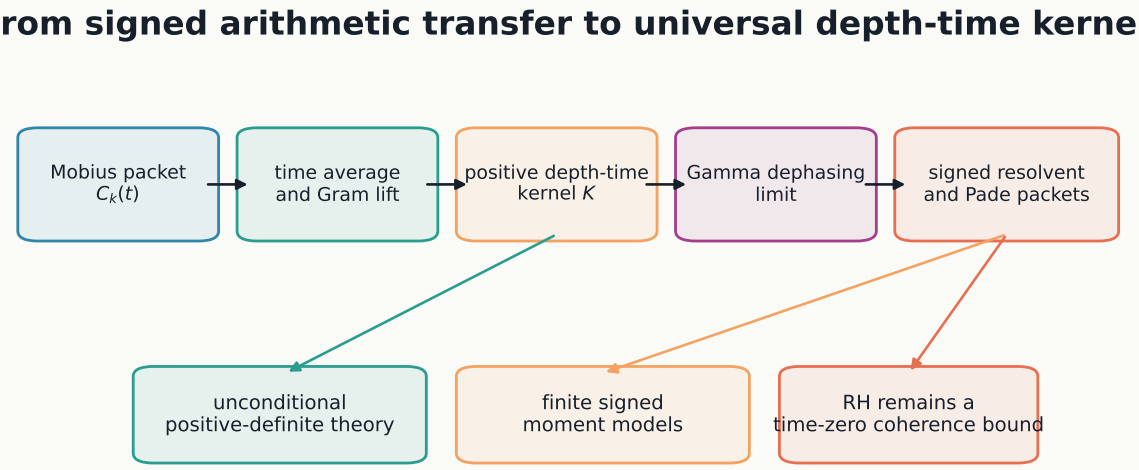



Figure 1.1: The mathematical pipeline. The signed transfer is lifted by time averaging into a positive kernel. The positive kernel has a universal Gamma limit. The signed resolvent and its finite Padé reductions preserve the coherent information that the positive average removes.

**Worked example 1.1** (Principal Bernoulli–zeta orbit)**.** Take $a = \beta = 2$ and the principal character. Then

$$C_{2,2}(k,0) = \sum_{j=0}^{k} (-1)^j \binom{k}{j} \frac{1}{\zeta(2j+2)}.$$

The first values are

$$C_{2,2}(0,0) = 0.607927101854\ldots,$$
$$C_{2,2}(1,0) = -0.316011301068\ldots,$$
$$C_{2,2}(2,0) = -0.256997111725\ldots.$$

These are the Báez-Duarte coefficients in the normalization used throughout the series.

Chapter 2

# The depth–time kernel

For two characters $\chi$ and $\psi$, define

$$\boxed{\mathcal{K}_{\chi,\psi}(k,\ell;\tau) = \sum_{n\geq 1} \mu(n)^2 \chi(n)\overline{\psi(n)} n^{-2a-\mathrm{i}\tau} y_n^{k+\ell}.}$$

When $\chi = \psi$, the weights are nonnegative.

**Theorem 2.1** (Exact time-average identity)**.** *For $a > 1$,*

$$\boxed{\lim_{T\to\infty} \frac{1}{2T} \int_{-T}^{T} C_{a,\beta,\chi}(k,t+\tau)\overline{C_{a,\beta,\psi}(\ell,t)}\,\mathrm{d}t = \mathcal{K}_{\chi,\psi}(k,\ell;\tau).}$$

*Proof.* Expand the product into the absolutely convergent double series. The time average of the factor

$$\mathrm{e}^{-\mathrm{i}t(\log n-\log m)}$$

is one when $n = m$ and tends to zero when $n \neq m$. Only the diagonal terms remain. □

**Theorem 2.2** (Positive-definite packet kernel)**.** *For one fixed character $\chi$, the function*

$$((k,t),(\ell,s)) \longmapsto \mathcal{K}_{\chi,\chi}(k,\ell;t-s)$$

*is positive definite.*

*Proof.* Define the feature vector

$$v_{k,t}(n) = \frac{\mu(n)\,|\chi(n)|}{n^a} y_n^k n^{-\mathrm{i}t}.$$

Then

$$\mathcal{K}_{\chi,\chi}(k,\ell;t-s) = \sum_n v_{k,t}(n)\overline{v_{\ell,s}(n)}.$$

Every finite Gram sum is therefore nonnegative. □

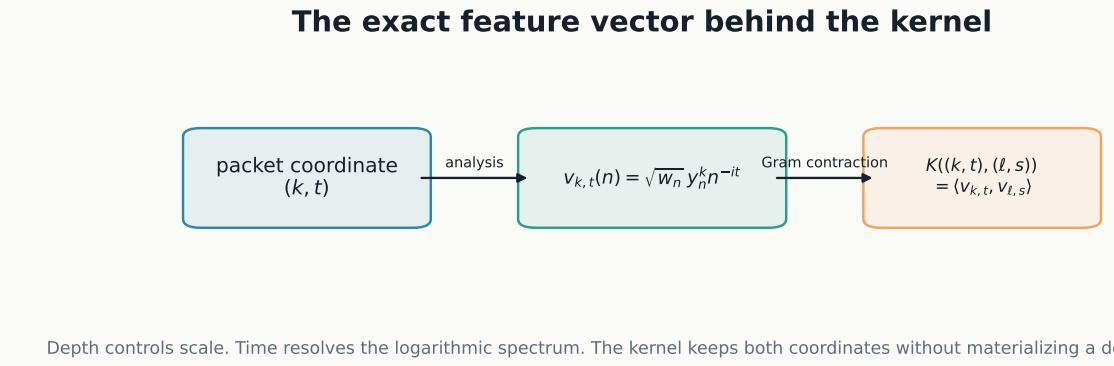



Figure 2.1: The exact feature map. Depth controls the active scale. Logarithmic time rotates each number-basis packet at frequency $\log n$. Their Gram contraction is the depth-time kernel.

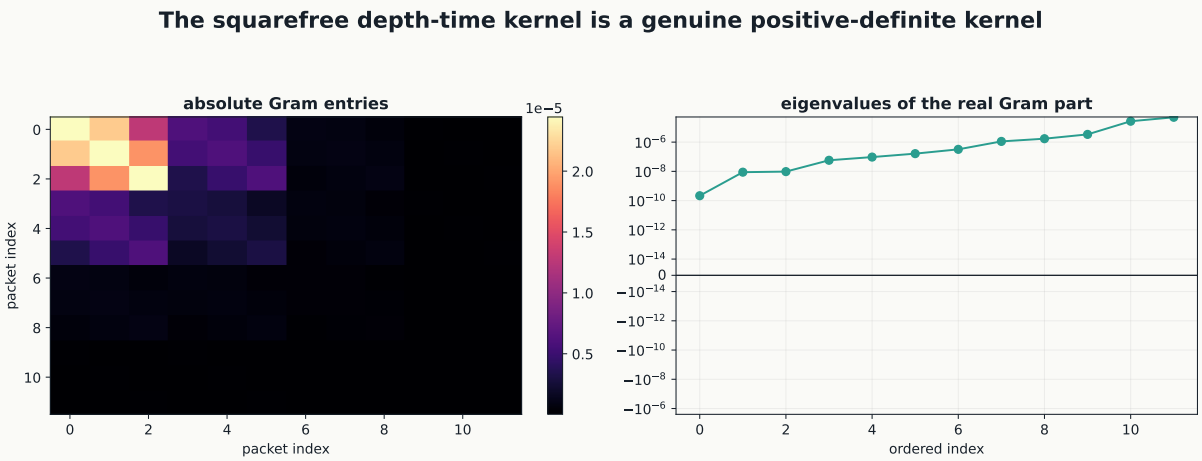



Figure 2.2: A finite depth-time Gram matrix. The matrix is Hermitian positive semidefinite. The plotted eigenvalues provide a numerical certificate for the displayed packet set.

Chapter 3

# Squarefree Dirichlet coordinates

Let

$$\xi = \chi\overline{\psi}$$

and define

$$D_\xi(s) = \sum_{n\geq 1} \frac{\mu(n)^2\xi(n)}{n^s}.$$

For $\Re s > 1$,

$$\boxed{D_\xi(s) = \prod_p \left(1 + \frac{\xi(p)}{p^s}\right) = \frac{L(s,\xi)}{L(2s,\xi^2)}.}$$

**Theorem 3.1** (Finite-difference coordinate formula)**.** *Let $m = k + \ell$. Then*

$$\boxed{\mathcal{K}_{\chi,\psi}(k,\ell;\tau) = \sum_{j=0}^{m} (-1)^j \binom{m}{j} D_\xi(2a + \beta j + \mathrm{i}\tau).}$$

*Proof.* Expand

$$y_n^m = \left(1 - n^{-\beta}\right)^m = \sum_{j=0}^{m} (-1)^j \binom{m}{j} n^{-\beta j}$$

and interchange the finite sum with the absolutely convergent Dirichlet series. □

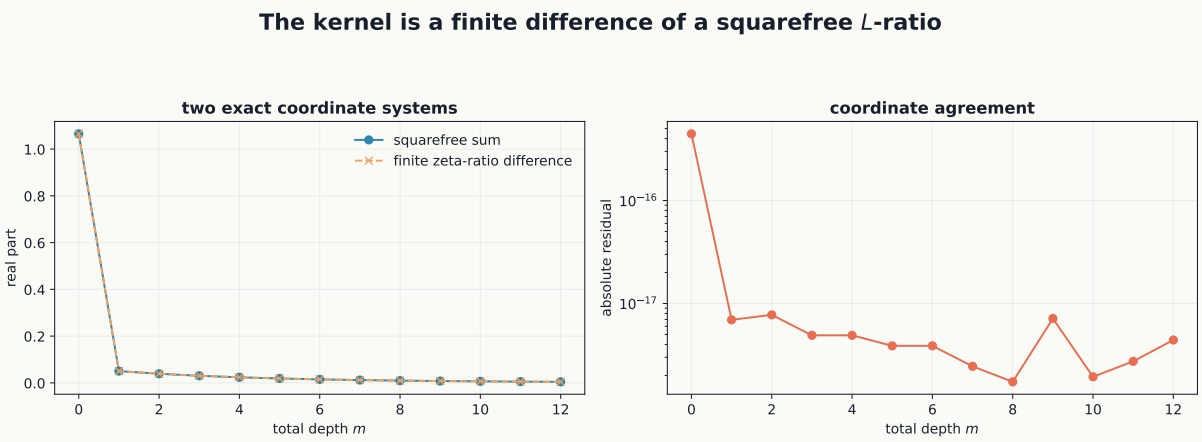



Figure 3.1: Two exact coordinates for one kernel. The direct squarefree sum and the finite difference of zeta ratios agree to floating-point roundoff in the worked calculation.

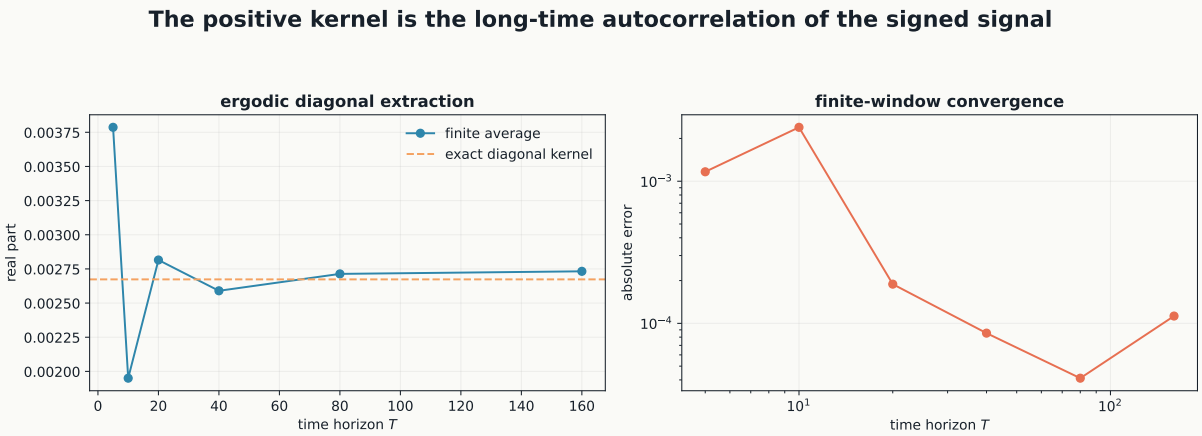



Figure 3.2: Finite-time extraction of the diagonal kernel. Increasing the observation window suppresses off-diagonal logarithmic frequencies and approaches the exact positive packet correlation.

# Part II

# Universal depth-time asymptotics

Chapter 4

# A complex Beta asymptotic

Assume $\chi = \psi$ has conductor $q$. Then

$$|\chi(n)|^2 = \mathbf{1}_{(n,q)=1}.$$

Let

$$A_q(X) = \sum_{\substack{n \le X \\ (n,q)=1}} \mu(n)^2.$$

The squarefree counting law gives

$$A_q(X) = \delta_q X + O_q(X^{1/2}),$$

where

$$\delta_q = \frac{1}{\zeta(2)} \prod_{p \mid q} \frac{p}{p+1}.$$

Set

$$\alpha = \frac{2a-1}{\beta}$$

and

$$\lambda(\tau) = \alpha + \frac{\mathrm{i}\tau}{\beta}.$$

**Theorem 4.1** (Uniform complex packet asymptotic)**.** *For every fixed $T > 0$, uniformly for $|\tau| \le T$,*

$$\boxed{\mathcal{K}_{\chi,\chi}(k,\ell;\tau) = \frac{\delta_q}{\beta} B(\lambda(\tau), k+\ell+1) + O_{a,\beta,q,T}\left((k+\ell)^{-(2a-1/2)/\beta}\right).}$$

*Consequently,*

$$\boxed{\mathcal{K}_{\chi,\chi}(k,\ell;\tau) \sim \frac{\delta_q}{\beta} \Gamma\left(\alpha + \frac{\mathrm{i}\tau}{\beta}\right) (k+\ell)^{-\alpha - \mathrm{i}\tau/\beta}.}$$

*Proof.* Apply partial summation to the function

$$f_m(x) = x^{-2a-\mathrm{i}\tau} \left(1 - x^{-\beta}\right)^m, \qquad m = k+\ell.$$

The main term is

$$\delta_q \int_1^\infty f_m(x)\, \mathrm{d}x.$$

With $u = x^{-\beta}$, this becomes

$$\frac{\delta_q}{\beta} \int_0^1 u^{\lambda(\tau)-1} (1-u)^m \, \mathrm{d}u = \frac{\delta_q}{\beta} B(\lambda(\tau), m+1).$$

The square-root error in $A_q(X)$, together with one derivative of $f_m$, contributes

$$O(m^{-(2a-1/2)/\beta})$$

uniformly on compact $\tau$ intervals. The Gamma asymptotic follows from the standard Beta quotient. □

Chapter 5

# The Gamma dephasing function

Define

$$\boxed{\mathcal{D}_{\alpha,\beta}(\tau) = \frac{\Gamma(\alpha + \mathrm{i}\tau/\beta)}{\Gamma(\alpha)}.}$$

This is the characteristic function of $\beta^{-1} \log U$ when

$$U \sim \mathrm{Gamma}(\alpha, 1).$$

It is therefore positive definite. Since the Gamma function has no zeros,

$$\boxed{\mathcal{D}_{\alpha,\beta}(\tau) \neq 0 \quad \text{for every real } \tau.}$$

**Corollary 5.1** (Equal-depth dephasing)**.** *Let*

$$A_k(\tau) = \frac{\mathcal{K}_{\chi,\chi}(k, k; \tau)}{\mathcal{K}_{\chi,\chi}(k, k; 0)}.$$

*Then*

$$\boxed{(2k)^{\mathrm{i}\tau/\beta} A_k(\tau) \longrightarrow \mathcal{D}_{\alpha,\beta}(\tau)}$$

*uniformly on compact real $\tau$ intervals.*

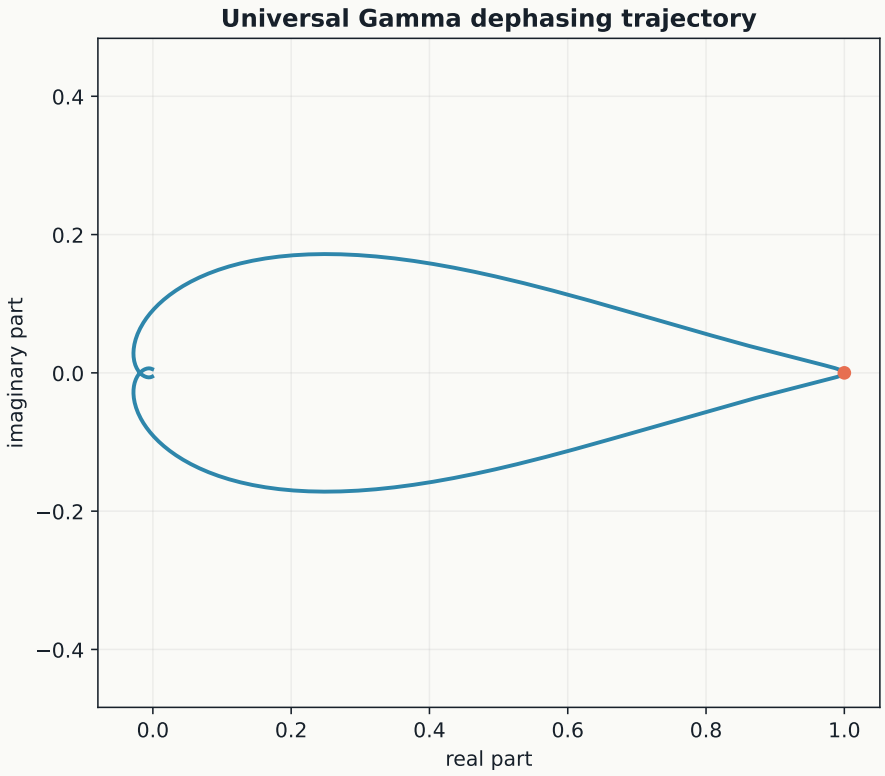



Figure 5.1: The limiting dephasing trajectory. The Gamma ratio traces a zero-free curve in the complex plane. The marked point is the normalized value at zero lag.

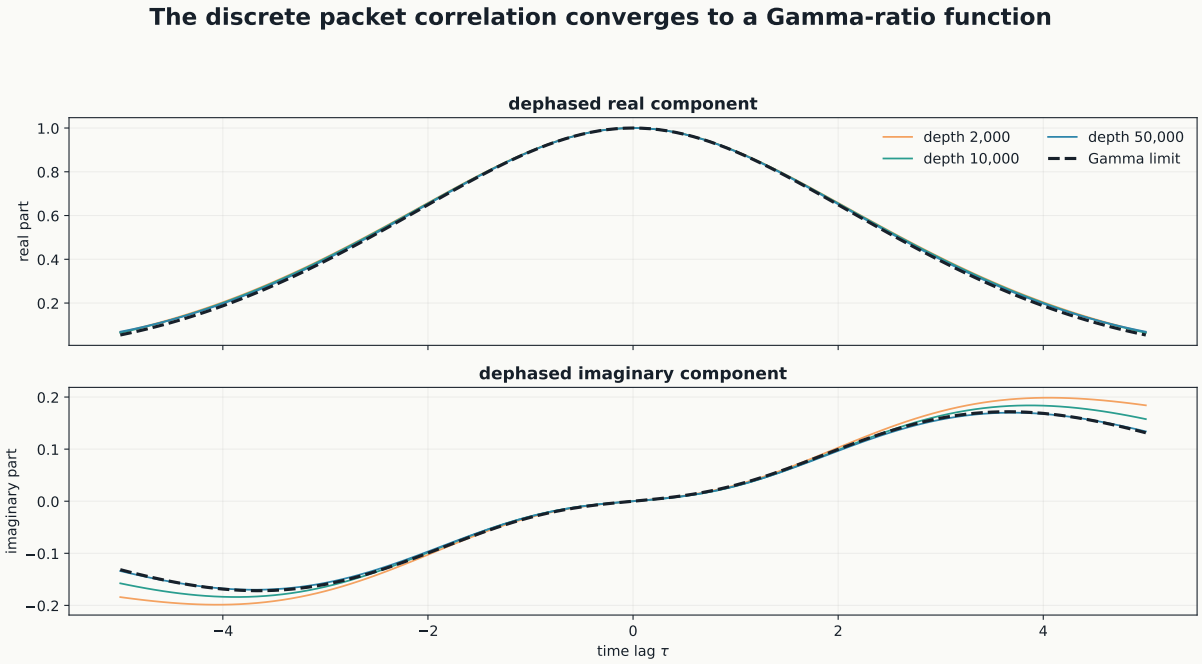



Figure 5.2: Convergence to the Gamma ratio. The dephased discrete correlations approach the same real and imaginary limit as the packet depth grows.

**Worked example 5.2** (Closed principal envelope)**.** For

$$a = \beta = 2, \qquad \alpha = \frac{3}{2},$$

we have

$$\mathcal{D}_{3/2,2}(\tau) = \frac{\Gamma(3/2 + \mathrm{i}\tau/2)}{\Gamma(3/2)}.$$

Using

$$|\Gamma(1/2 + \mathrm{i}y)|^2 = \frac{\pi}{\cosh(\pi y)},$$

one obtains

$$\boxed{\left|\mathcal{D}_{3/2,2}(\tau)\right|^2 = \frac{1+\tau^2}{\cosh(\pi\tau/2)}.}$$

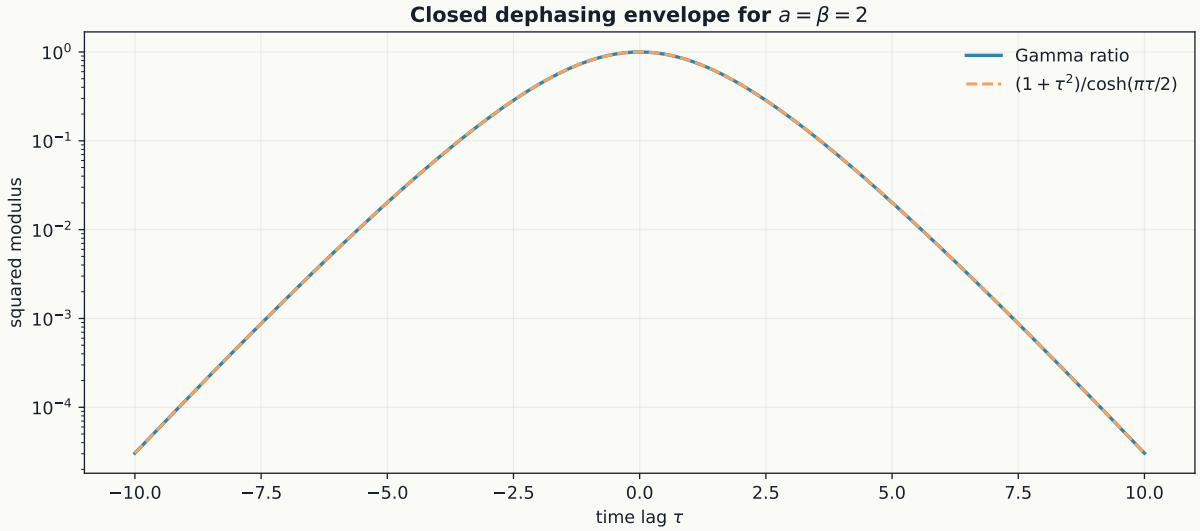



Figure 5.3: The principal closed-form dephasing envelope. The Gamma-ratio modulus and the elementary hyperbolic-cosine expression coincide throughout the displayed range.

The phase derivative also has a closed law:

$$\boxed{\frac{\mathrm{d}}{\mathrm{d}\tau}\arg\mathcal{D}_{\alpha,\beta}(\tau) = \frac{1}{\beta}\Re\psi\left(\alpha + \frac{\mathrm{i}\tau}{\beta}\right).}$$

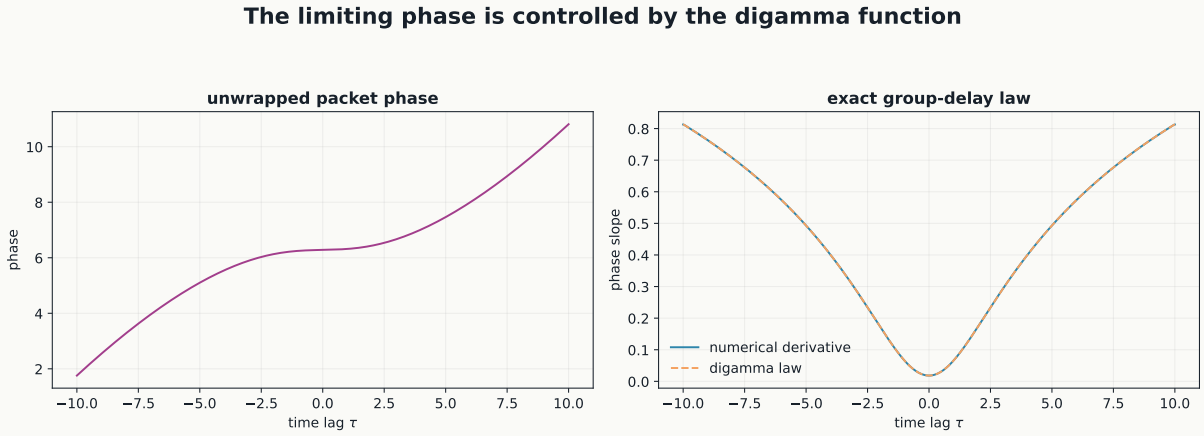



Figure 5.4: The phase and its derivative. The digamma function supplies the exact group-delay law of the limiting packet correlation.

Table 5.1: Selected values of the limiting Gamma dephasing function for $a = \beta = 2$.

| $\tau$ | Real part | Imaginary part | Squared modulus |
|---|---|---|---|
| -4 | 0.18721515 | -0.16865147 | 0.06349283 |
| -2 | 0.64917367 | -0.09953509 | 0.43133369 |
| -1 | 0.89225332 | -0.03094590 | 0.79707363 |
| 0 | 1.00000000 | 0.00000000 | 1.00000000 |
| 1 | 0.89225332 | 0.03094590 | 0.79707363 |
| 2 | 0.64917367 | 0.09953509 | 0.43133369 |
| 4 | 0.18721515 | 0.16865147 | 0.06349283 |

Chapter 6

# Log-frequency cumulants

Normalize the depth-$m$ positive weights by

$$\mathbb{P}_m(n) = \frac{\mu(n)^2 \, |\chi(n)|^2 \, n^{-2a} y_n^m}{\mathcal{K}_{\chi,\chi}(m, 0; 0)}.$$

Define the centered logarithmic frequency

$$Z_m = \log n - \frac{1}{\beta} \log m.$$

The preceding characteristic-function limit gives

$$Z_m \Longrightarrow -\frac{1}{\beta} \log U, \qquad U \sim \text{Gamma}(\alpha, 1).$$

**Corollary 6.1** (Polygamma cumulants)**.** *For every fixed order for which moment convergence is taken,*

$$\boxed{\kappa_r(Z_m) \longrightarrow \frac{(-1)^r}{\beta^r} \psi_{r-1}(\alpha).}$$

*In particular,*

$$\boxed{\mathbb{E}Z_m \longrightarrow -\frac{\psi(\alpha)}{\beta}}$$

*and*

$$\boxed{\mathrm{Var}(Z_m) \longrightarrow \frac{\psi_1(\alpha)}{\beta^2}.}$$

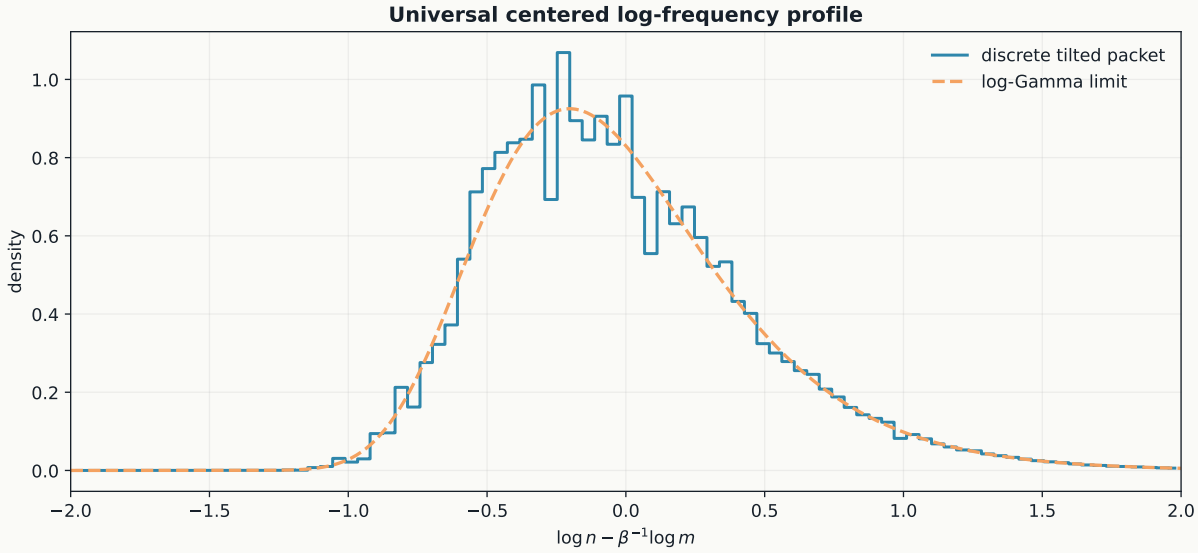



Figure 6.1: The centered logarithmic spectrum. The finite squarefree packet histogram approaches the log-Gamma density predicted by the dephasing function.

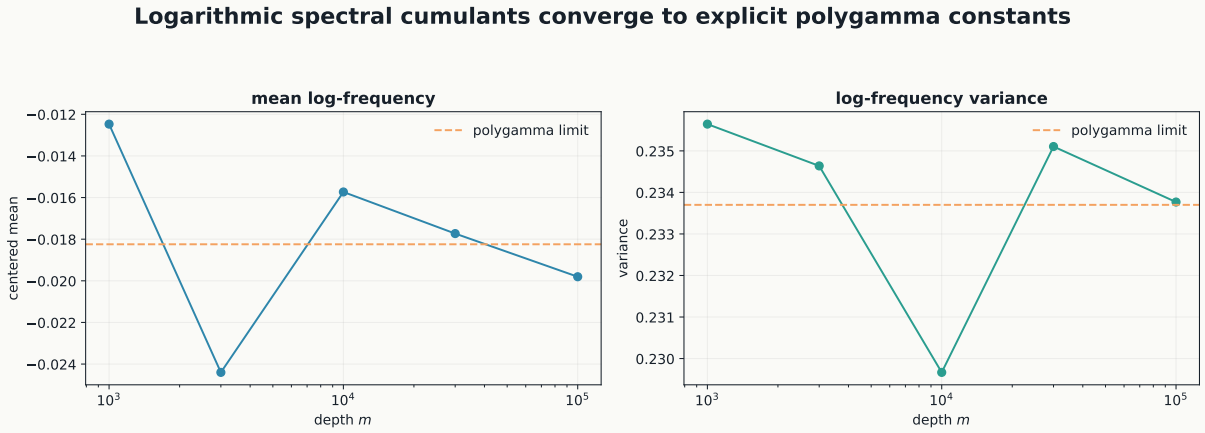



Figure 6.2: Convergence of the first two cumulants. The mean and variance settle toward explicit digamma and trigamma constants.

Chapter 7

# The cross-depth overlap law

At zero time lag, define

$$\mathcal{O}(k,\ell) = \frac{\mathcal{K}_{\chi,\chi}(k,\ell;0)}{\sqrt{\mathcal{K}_{\chi,\chi}(k,k;0)\mathcal{K}_{\chi,\chi}(\ell,\ell;0)}}.$$

**Theorem 7.1** (Universal scale overlap)**.** *If $k,\ell \to \infty$ and $k/\ell \to r \in (0,\infty)$, then*

$$\boxed{\mathcal{O}(k,\ell) \longrightarrow \left(\frac{2\sqrt{r}}{1+r}\right)^{\alpha} = \operatorname{sech}^{\alpha}\left(\frac{1}{2}\log r\right).}$$

*Proof.* The zero-lag kernel is regularly varying:

$$\mathcal{K}(k,\ell;0) \sim A(k+\ell)^{-\alpha}.$$

The two diagonal norms satisfy

$$\mathcal{K}(k,k;0) \sim A(2k)^{-\alpha}$$

and the analogous formula for $\ell$. Taking the ratio gives the result. □

**Worked example 7.2** (A four-to-one depth ratio)**.** For $a = \beta = 2$, we have $\alpha = 3/2$. When $k/\ell = 4$,

$$\mathcal{O}(k,\ell) \longrightarrow \left(\frac{4}{5}\right)^{3/2} = 0.7155417528\ldots.$$

The finite calculation at depths $k = 40000$ and $\ell = 10000$ gives a value within two percent of this limit.

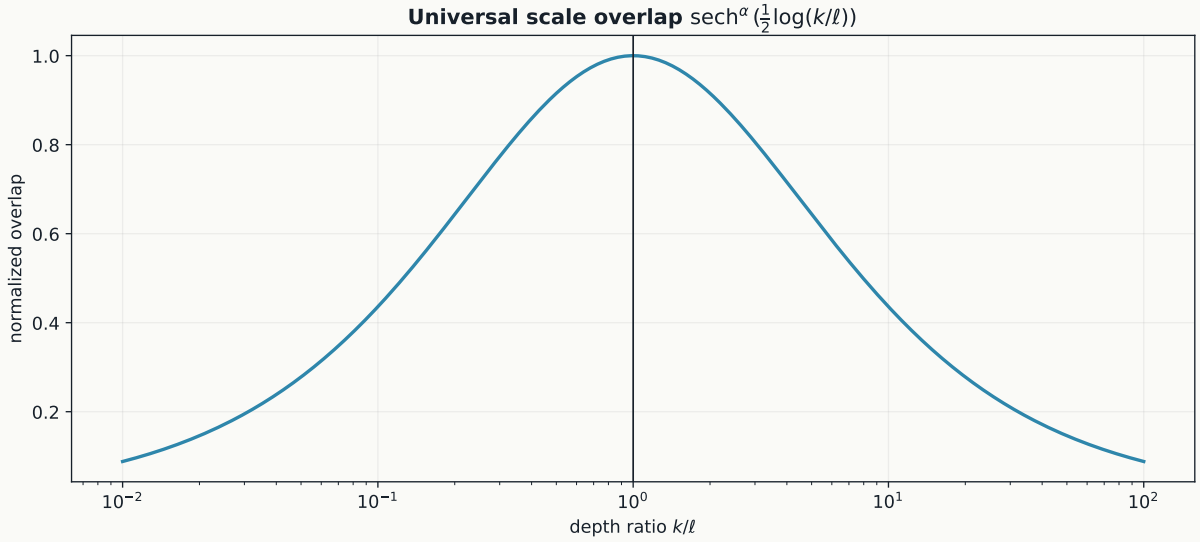



Figure 7.1: The exact limiting overlap profile. Packets at comparable depths retain substantial overlap. Packets separated multiplicatively in depth become asymptotically orthogonal.

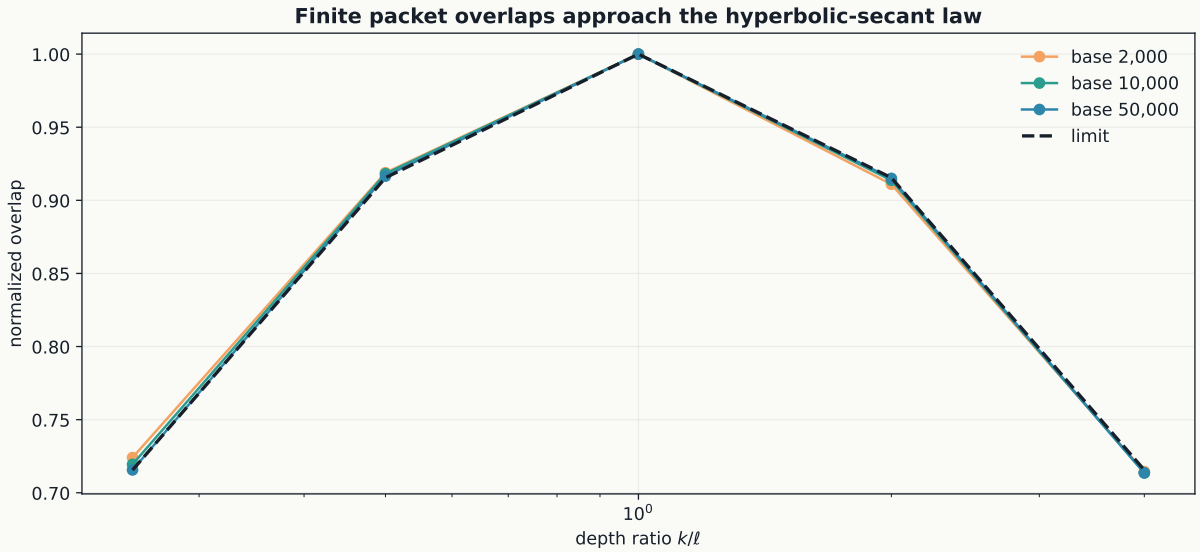



Figure 7.2: Finite-depth convergence. Several absolute scales approach the same ratio-dependent law.

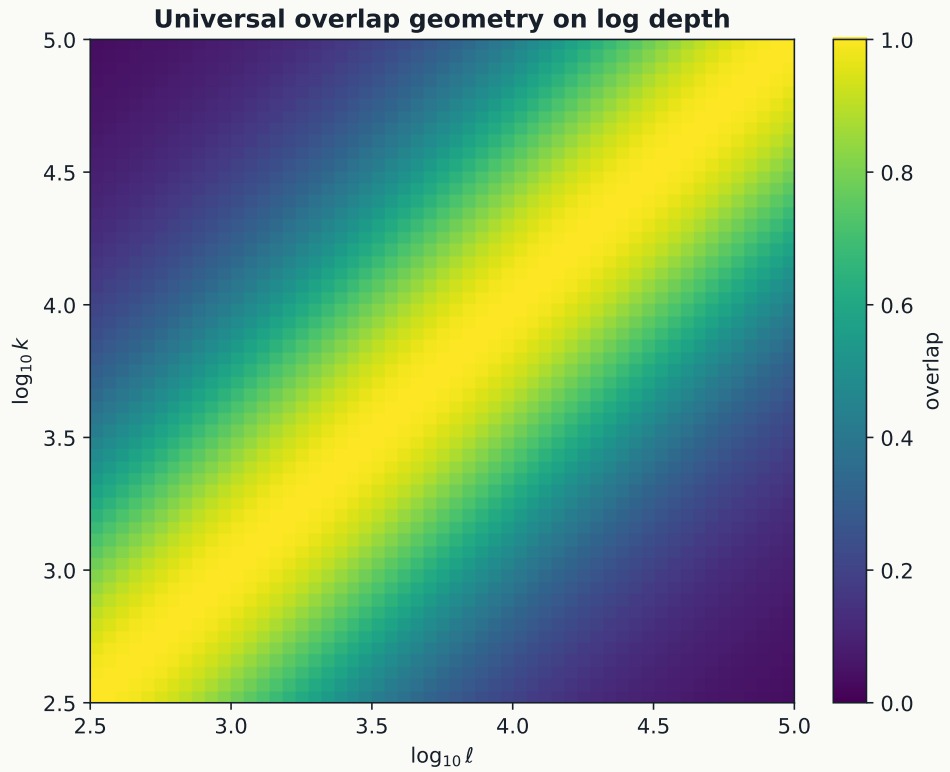



Figure 7.3: The geometry on logarithmic depth. The overlap depends asymptotically only on the difference of log depths, which produces a translation-invariant ridge in scale coordinates.

# Part III

# Character packets and arithmetic decoherence

## Chapter 8

## Cross-character depth-time kernels

For two characters modulo the same conductor, the cross-kernel has the exact time-average interpretation already proved. If

$$\xi = \chi\overline{\psi}$$

is nonprincipal, its squarefree partial sums satisfy

$$\sum_{n\le X} \mu(n)^2\xi(n) = O_q(X^{1/2}).$$

This follows from

$$\mu(n)^2 = \sum_{d^2\mid n} \mu(d)$$

and the bounded complete-period sums of a nonprincipal character.

**Theorem 8.1** (Character depth-time decoherence)**.** *If $\chi \neq \psi$, then uniformly for $|\tau| \leq T$,*

$$\boxed{\mathcal{K}_{\chi,\psi}(k,\ell;\tau) = O_{a,\beta,q,T}\left((k+\ell)^{-(2a-1/2)/\beta}\right).}$$

*After diagonal normalization,*

$$\boxed{\frac{\mathcal{K}_{\chi,\psi}(k,\ell;\tau)}{\sqrt{\mathcal{K}_{\chi,\chi}(k,k;0)\mathcal{K}_{\psi,\psi}(\ell,\ell;0)}} = O\left((k+\ell)^{-1/(2\beta)}\right)}$$

*whenever the two depths remain comparable.*

*Proof.* Use partial summation with the nonprincipal square-root bound instead of the principal density term. The same scaled derivative estimate used in the complex Beta theorem yields the stated power. Dividing by the two diagonal norms removes $(k+\ell)^{-\alpha}$ and leaves the factor $(k+\ell)^{-1/(2\beta)}$. □

For $a = \beta = 2$, the unconditional normalized decay rate is

$$(k+\ell)^{-1/4}.$$

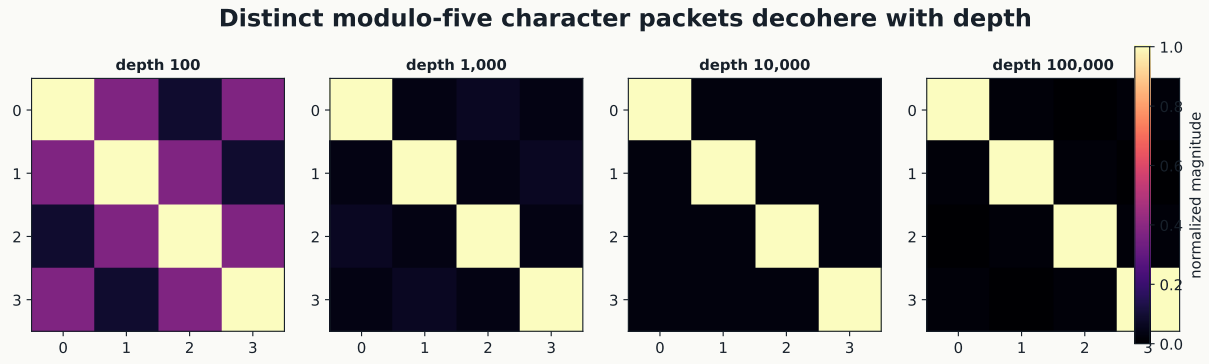



Figure 8.1: Modulo-five character Gram matrices. The diagonal remains normalized to one while the off-diagonal sectors weaken with increasing depth.

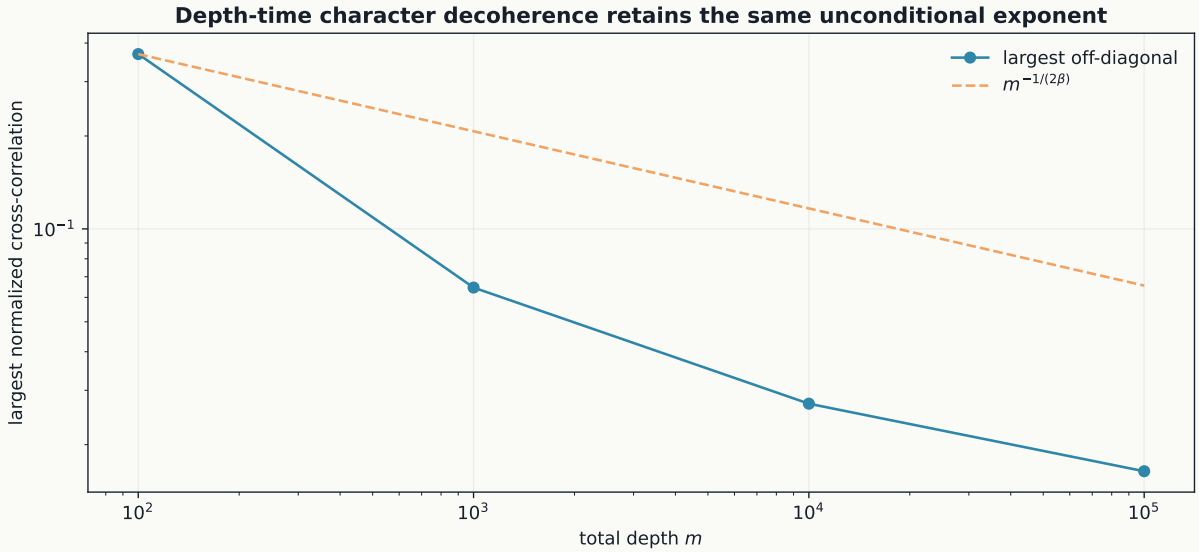



Figure 8.2: The largest off-diagonal character correlation. The finite values decrease in the direction predicted by the unconditional quarter-power law.

# Part IV

# The signed Möbius function family

## Chapter 9

## The Möbius resolvent

Define the signed resolvent

$$\boxed{\mathcal{C}_{a,\beta,\chi}(z,t) = \sum_{k=0}^{\infty} C_{a,\beta,\chi}(k,t) z^k.}$$

For $|z| < 1$, geometric summation gives

$$\boxed{\mathcal{C}_{a,\beta,\chi}(z,t) = \sum_{n\geq 1} \frac{\mu(n)\chi(n) n^{-a-\mathrm{i}t}}{1 - z(1-n^{-\beta})}.}$$

Since $a > 1$, this sum extends locally uniformly to

$$\mathbb{C} \setminus [1, \infty).$$

Unlike the positive Stieltjes resolvent of Paper III, the signed resolvent is not asserted to be zero-free.

In the smaller domain where the following geometric expansion converges,

$$\boxed{\mathcal{C}_{a,\beta,\chi}(z,t) = \frac{1}{1-z} \sum_{j=0}^{\infty} \left(-\frac{z}{1-z}\right)^j \frac{1}{L(a+\beta j + \mathrm{i}t, \chi)}.}$$

**Theorem 9.1** (Parameter-shift equation)**.** *The signed resolvent satisfies*

$$\boxed{(1-z)\mathcal{C}_{a,\beta,\chi}(z,t)+z\mathcal{C}_{a+\beta,\beta,\chi}(z,t)=\frac{1}{L(a+\mathrm{i}t,\chi)}.}$$

*Proof.* Multiply the depth recursion by $z^k$ and sum over $k \geq 0$. The left shift contributes

$$\frac{\mathcal{C}_a - C_a(0,t)}{z}.$$

Since

$$C_a(0,t) = L(a+\mathrm{i}t,\chi)^{-1},$$

rearrangement gives the result. □

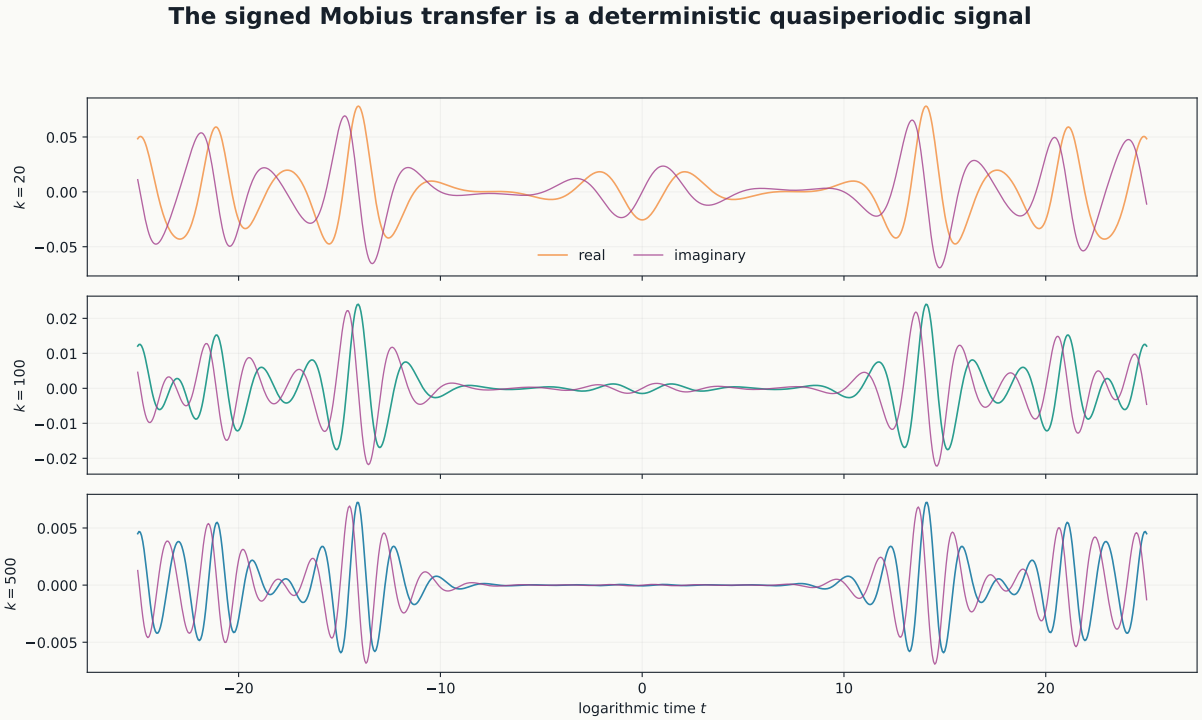



Figure 9.1: The signed transfer in logarithmic time. The signal is quasiperiodic because every integer packet rotates at its own frequency $\log n$. Increasing the depth changes the active scale but not the frequency law.

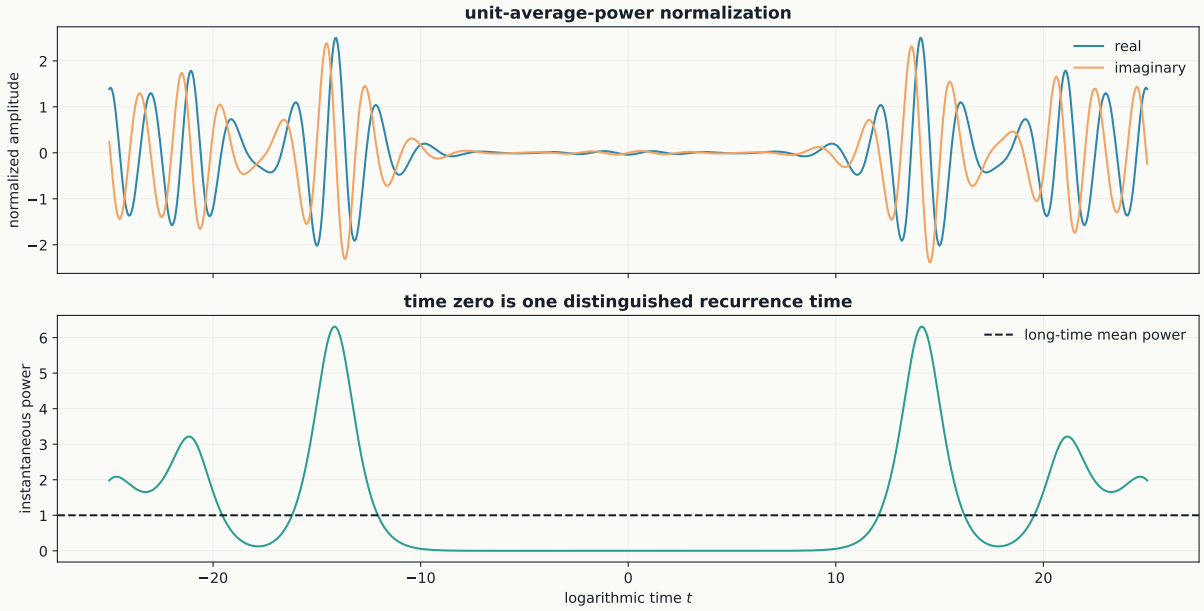



Figure 9.2: The unit-average-power process. The positive packet energy normalizes the long-time mean power to one. The value at $t = 0$ remains a distinguished coherent recurrence.

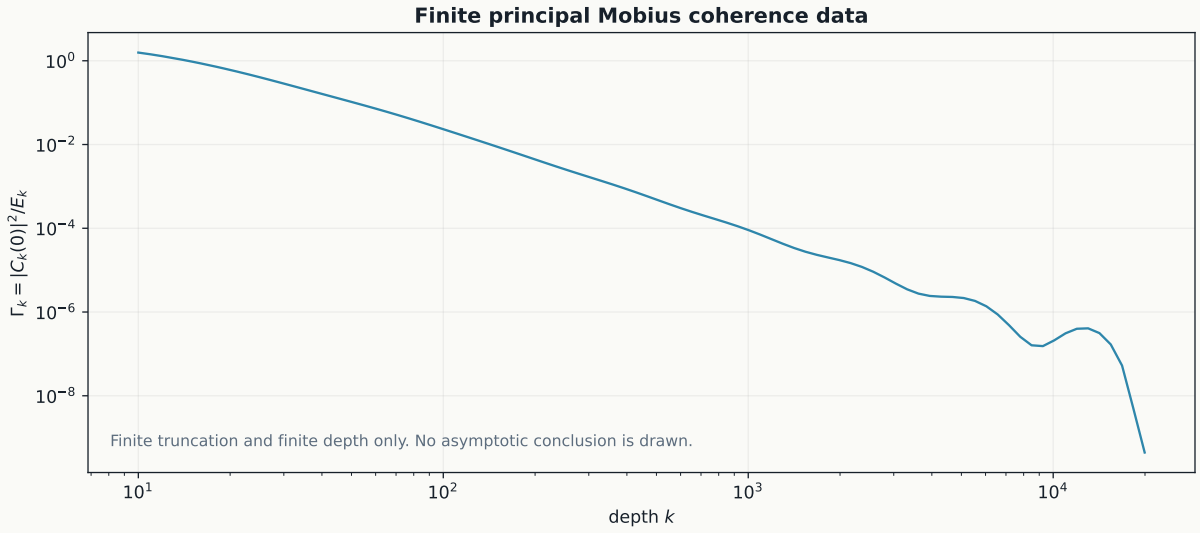



Figure 9.3: Finite time-zero coherence data. The plot is a finite computational record only. It is not used to infer the asymptotic RH bound.

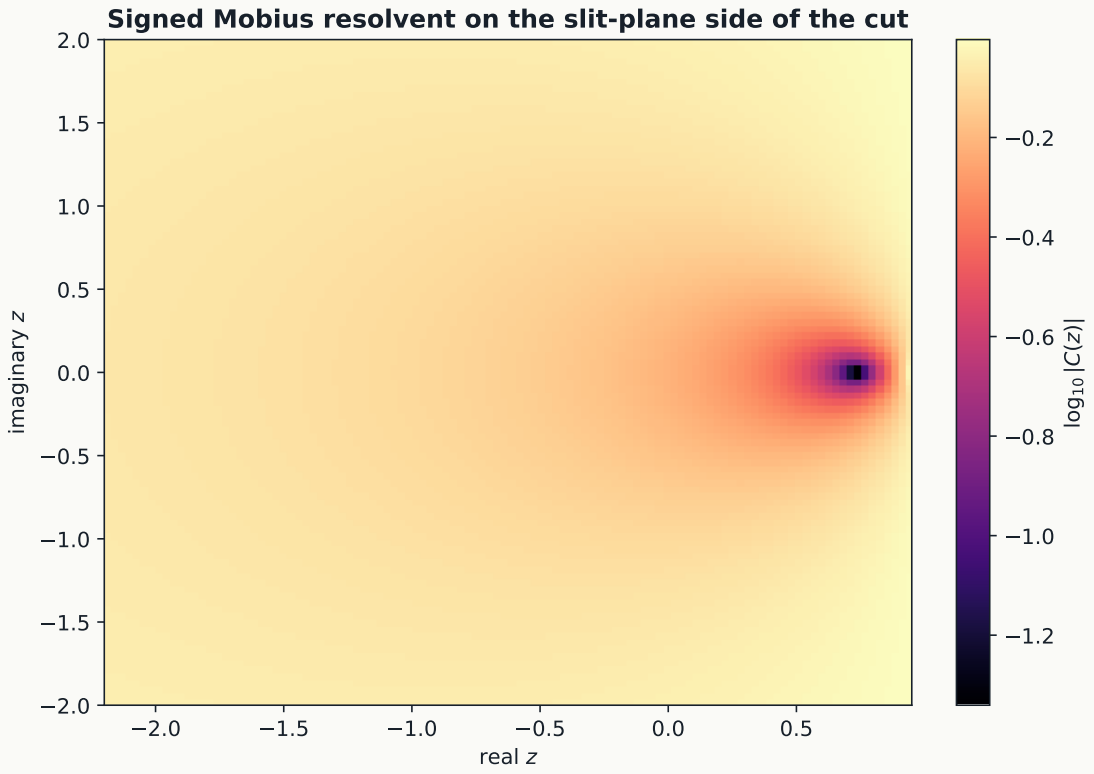



Figure 9.4: The signed resolvent. The function is analytic on the displayed side of the physical cut. Unlike the positive resolvent, it may have zeros and sign cancellations.

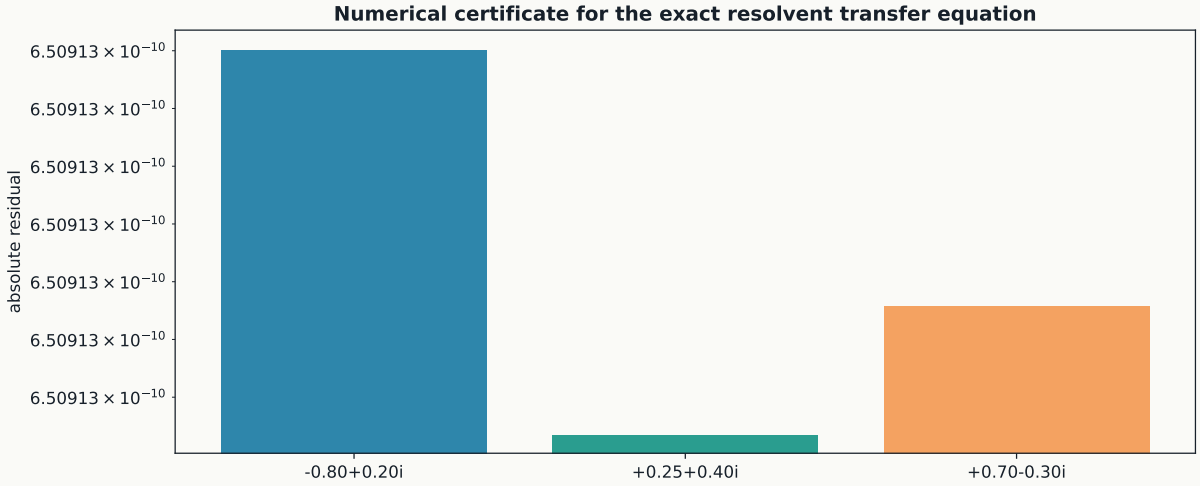



Figure 9.5: The transfer-equation residual. Three complex evaluation points verify the parameter-shift identity to the truncation accuracy of the number-basis sum.

Chapter 10

# The Borel–Riesz bridge

The exponential generating function of the signed depth sequence is

$$\sum_{k=0}^{\infty} C_{a,\beta,\chi}(k,t)\frac{x^k}{k!} = \sum_{n\geq 1} \frac{\mu(n)\chi(n)}{n^{a+it}} \mathrm{e}^{x(1-n^{-\beta})}.$$

Define

$$\mathcal{R}_{a,\beta,\chi}(x,t) = x \sum_{n\geq 1} \frac{\mu(n)\chi(n)}{n^{a+it}} \mathrm{e}^{-x/n^{\beta}}.$$

Then

$$\boxed{\sum_{k=0}^{\infty} C_{a,\beta,\chi}(k,t)\frac{x^k}{k!} = \frac{\mathrm{e}^x}{x}\mathcal{R}_{a,\beta,\chi}(x,t).}$$

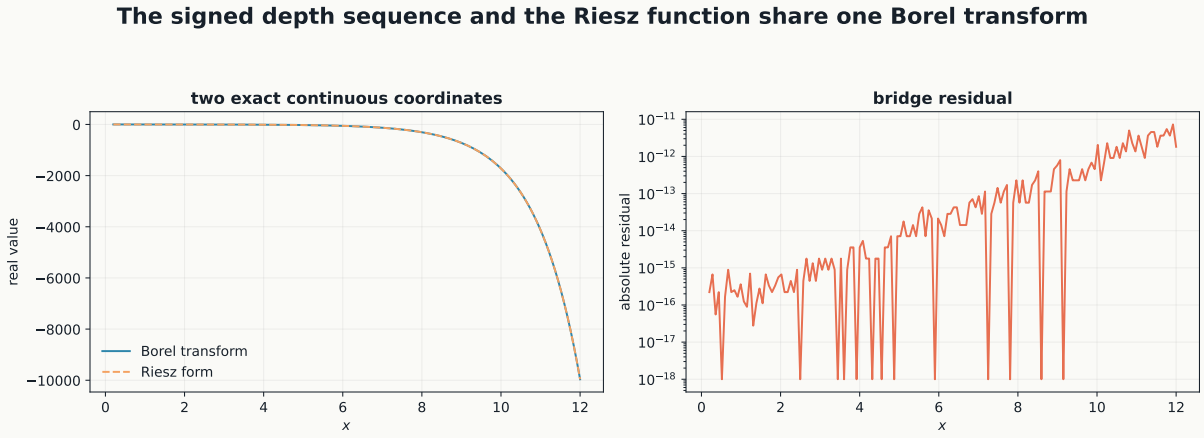



Figure 10.1: The Borel–Riesz identity. The discrete depth sequence and its continuous companion are two exact coordinates of the same Möbius transfer.

Chapter 11

# Signed Padé packet models

Let

$$c_j = C_{a,\beta,\chi}(j,t)$$

and define the Hankel determinant

$$\Delta_m(t) = \det[c_{i+j}]_{i,j=0}^{m-1}.$$

**Theorem 11.1** (Finite signed Padé reduction)**.** *Assume*

$$\Delta_m(t) \neq 0.$$

*Then there is a unique rational function*

$$\boxed{\mathcal{C}_m(z,t) = \frac{P_{m-1}(z,t)}{Q_m(z,t)}, \qquad Q_m(0,t) = 1,}$$

*such that*

$$\mathcal{C}_{a,\beta,\chi}(z,t) - \mathcal{C}_m(z,t) = O(z^{2m}).$$

*If the poles of $Q_m^{-1}$ are simple, then*

$$\boxed{\mathcal{C}_m(z,t) = \sum_{r=1}^{m} \frac{\lambda_r(t)}{1 - z\xi_r(t)}}$$

*and*

$$\boxed{C_{a,\beta,\chi}(j,t) = \sum_{r=1}^{m} \lambda_r(t)\xi_r(t)^j, \qquad 0 \leq j \leq 2m-1.}$$

*Proof.* Write

$$Q_m(z) = 1 + q_1 z + \cdots + q_m z^m.$$

The conditions on coefficients $z^m$ through $z^{2m-1}$ form a linear system whose matrix is a column reversal of the Hankel matrix $[c_{i+j}]$. The determinant assumption gives a unique denominator. The numerator is then the polynomial part of $Q_m\mathcal{C}$ through degree $m-1$. This proves the moment contract. Simple poles give the partial-fraction form. □

**Worked example 11.2** (The first signed packet leaves the physical interval)**.** For the principal case,

$$\xi_1 = \frac{c_1}{c_0} = -0.5198177546\ldots.$$

The positive packet spectrum belongs to $[0, 1)$. The first signed effective node is already negative. This shows that the signed reduction is an indefinite moment problem rather than a positive one.

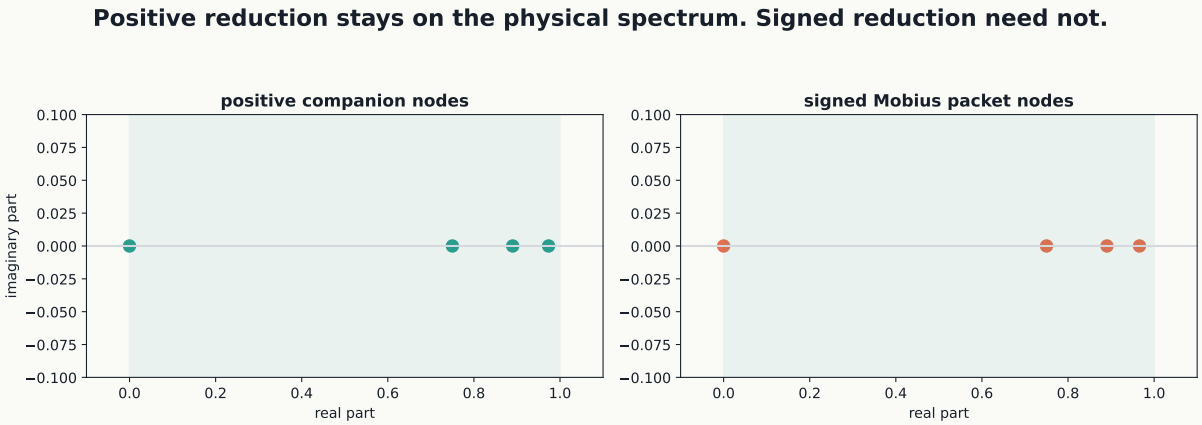



Figure 11.1: Positive and signed packet nodes. The positive companion reduction stays in the declared spectral interval. The signed Möbius reduction can leave it because cancellation replaces positivity.

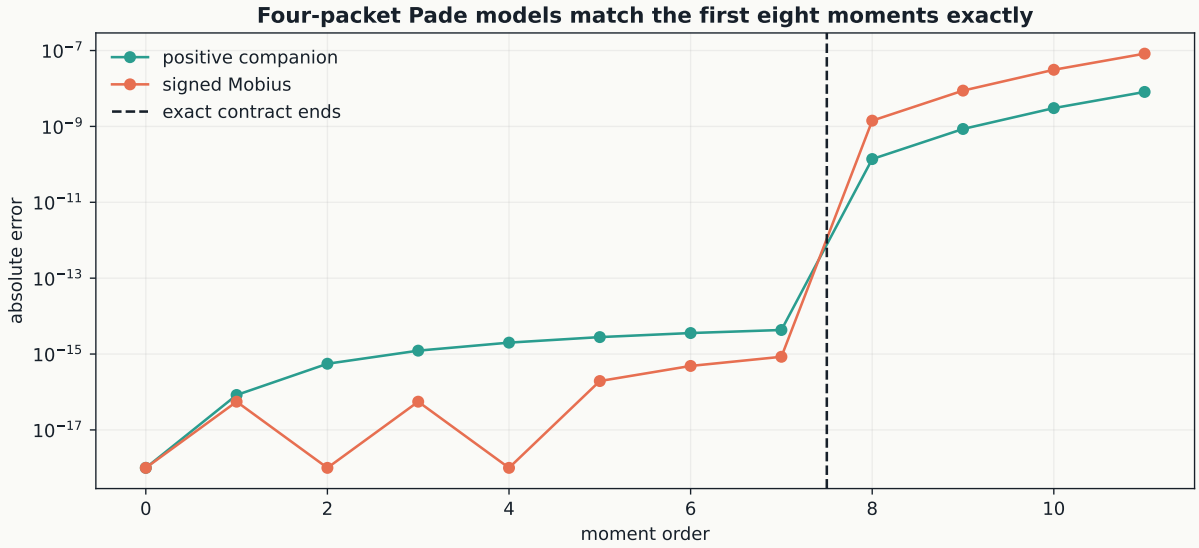



Figure 11.2: Moment-exact finite reduction. Four packets preserve the first eight positive or signed moments. The later moments are predictions rather than part of the exact contract.

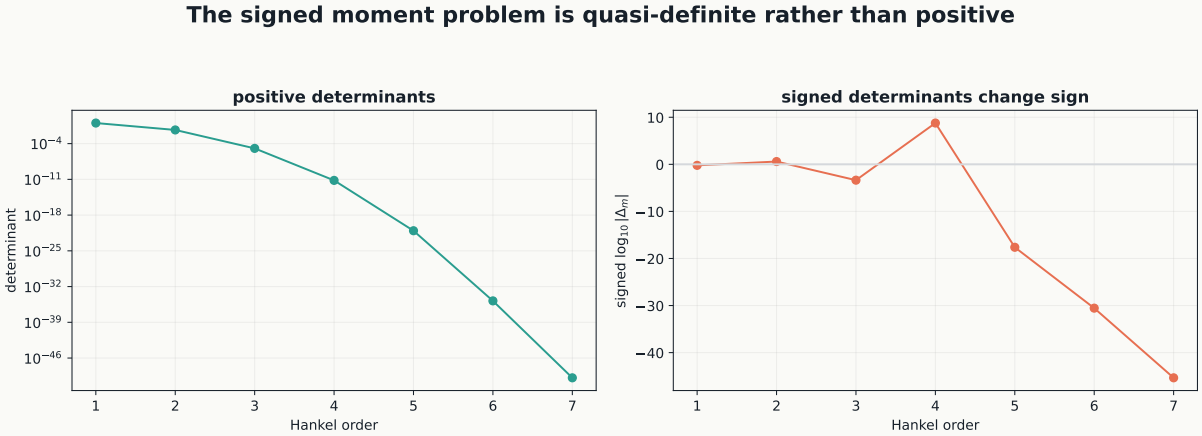



Figure 11.3: Positive and signed Hankel determinants. The positive determinants remain positive. The signed determinants change sign and become increasingly ill-conditioned, which exposes the nonbreakdown boundary directly.

Table 11.1: Four-packet Padé nodes and weights for the positive companion and signed Möbius sequences.

| Family | Node real | Node imag. | Weight real | Weight imag. |
|---|---|---|---|---|
| positive | 0.000000000 | 0.00e+00 | 1.000000003 | -2.17e-17 |
| positive | 0.750017741 | -6.47e-15 | 0.062527773 | -3.78e-15 |
| positive | 0.890070247 | 9.21e-15 | 0.012629961 | 6.89e-15 |
| positive | 0.973460741 | -1.13e-15 | 0.002770400 | -3.09e-15 |
| signed | -0.000000003 | -0.00e+00 | 0.999999972 | 1.68e-16 |
| signed | 0.750048271 | -1.13e-14 | -0.250305329 | 2.33e-14 |
| signed | 0.890460587 | 1.78e-14 | -0.114859170 | -5.74e-14 |
| signed | 0.965969069 | -2.28e-15 | -0.026908372 | 3.40e-14 |

# Part V

# Quantum and recurrence interpretation

## Chapter 12

## The number-basis operator system

Work in the number basis indexed by $n \geq 2$ and define

$$H\,|n\rangle = \log n\,|n\rangle\,,$$

$$K_\beta\,|n\rangle = -\log(1-n^{-\beta})\,|n\rangle\,,$$

and

$$\mathcal{M}_\chi\,|n\rangle = \mu(n)\chi(n)\,|n\rangle\,.$$

These three diagonal operators commute. Let

$$|\phi_a\rangle = \sum_{n\geq 2} n^{-a/2}\,|n\rangle\,.$$

Then the exact operator identity is

$$\boxed{C_{a,\beta,\chi}(k,t) = \mathbf{1}_{\{k=0\}} + \langle\phi_a|\,\mathcal{M}_\chi \mathrm{e}^{-kK_\beta}\mathrm{e}^{-\mathrm{i}tH}\,|\phi_a\rangle\,.}$$

The isolated term is the $n=1$ packet. It contributes at depth zero and vanishes for every positive depth. Keeping it separate avoids assigning an infinite value to $K_\beta$ at $n=1$. The external quantum construction that motivates this coordinate uses the same logarithmic Hamiltonian and number-basis phase. The additional Euclidean filter and Möbius sign observable are specific to the present packet system.

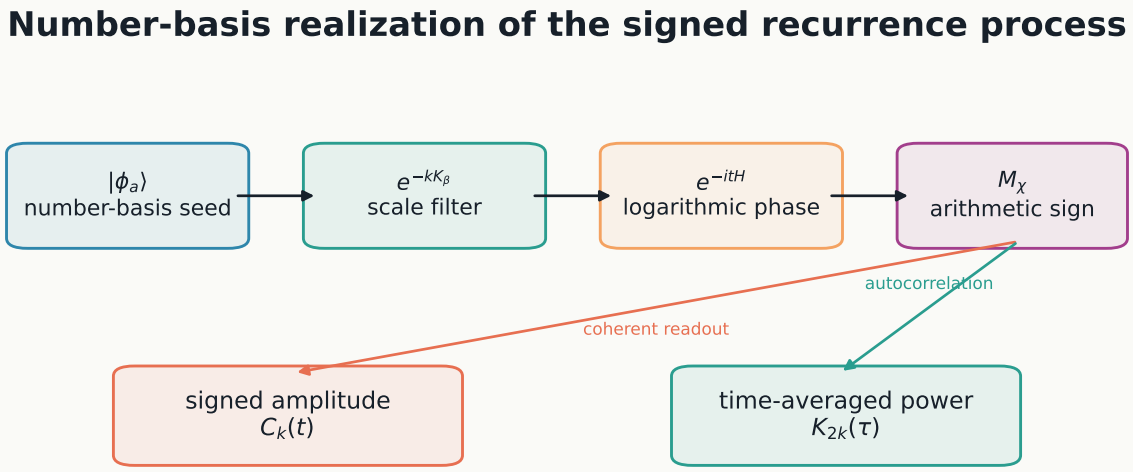



Figure 12.1: The operator realization. The signed amplitude is a coherent readout. Time averaging produces the positive depth-time kernel and removes only the off-diagonal logarithmic frequencies.

Chapter 13

# The recurrence process and the RH boundary

For the principal parameters define

$$c_k = C_{2,2}(k, 0)$$

and

$$E_k = \mathcal{K}(k, k; 0) = \sum_{n\geq 1} \frac{\mu(n)^2}{n^4} \left(1 - \frac{1}{n^2}\right)^{2k}.$$

The preceding papers prove

$$E_k \sim \frac{3}{2^{5/2}\pi^{3/2}} k^{-3/2}.$$

Define

$$\rho_k(t) = \frac{C_{2,2}(k, t)}{\sqrt{E_k}}.$$

The exact time-average identity gives

$$\boxed{\lim_{T\to\infty} \frac{1}{2T} \int_{-T}^{T} |\rho_k(t)|^2 \, \mathrm{d}t = 1.}$$

The time-zero power is

$$\Gamma_k = |\rho_k(0)|^2 = \frac{|c_k|^2}{E_k}.$$

The classical sequential criterion and the proved energy asymptotic give

$$\boxed{\mathrm{RH} \iff \Gamma_k = O_\varepsilon(k^\varepsilon) \quad \text{for every } \varepsilon > 0.}$$

The positive autocorrelation is therefore fully controlled. The remaining assertion is that the deterministic recurrence time $t = 0$ is not polynomially exceptional relative to the unit average power.

*Boundary statement* 13.1 (The exact remaining obstruction). Gamma dephasing, scale overlap, character decoherence, positive Gram structure, and finite signed Padé reduction do not prove the principal coherence bound. The missing theorem must use the internal arithmetic signs of $\mu(n)$ inside the principal sector.

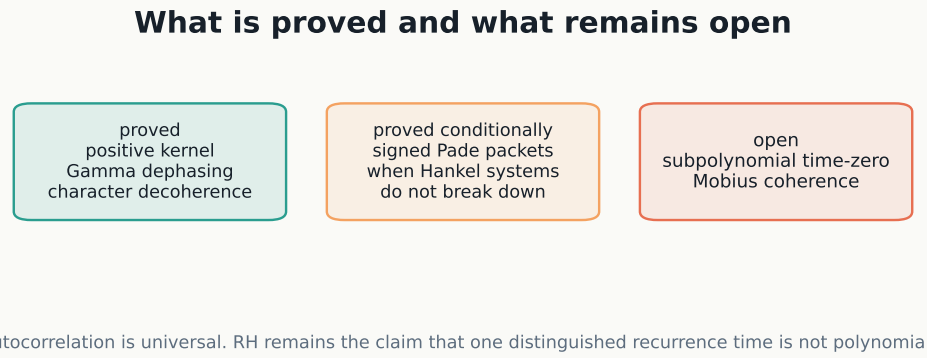



Figure 13.1: The exact boundary. The positive depth-time theory is unconditional. Finite signed models are exact under a visible nonbreakdown condition. The principal time-zero coherence remains open.

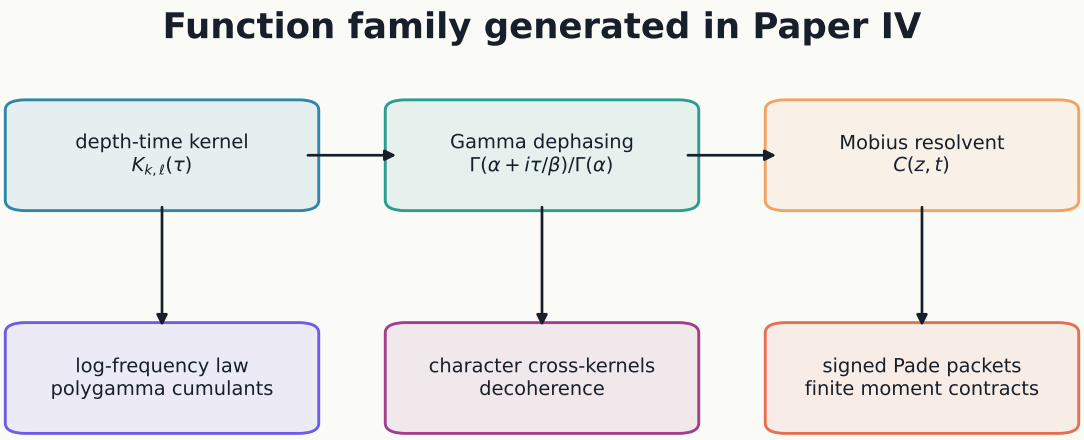



Figure 13.2: The generated function family. The paper adds one positive reproducing kernel, one zero-free Gamma dephasing limit, one signed resolvent, one log-frequency law, one character cross-kernel family, and finite signed Padé packet models.

# Part VI

# Worked algorithms and verification

## Chapter 14

## Computational route

**Algorithm 14.1** (Constructing the depth-time kernel and signed reduction)**.** 1. Sieve the Möbius function through the declared cutoff.

2. Form the nodes $y_n = 1 - n^{-\beta}$.
3. Form the positive weights $\mu(n)^2 \, |\chi(n)|^2 \, n^{-2a}$.
4. Evaluate the depth-time kernel by diagonal contraction.
5. Evaluate the signed transfer with the Möbius phase retained.
6. Compare the dephased positive kernel with the Gamma-ratio limit.
7. Form signed moments $c_0, \dots, c_{2m-1}$.
8. Check the Hankel determinant.
9. Solve the Padé denominator system when the determinant is nonzero.
10. Return the $m$ signed packet nodes and weights with a $2m$-moment certificate.

Chapter 15

# Verification ledger

18/18 built-in checks passed

Table 15.1: Built-in computational certificates. Full numerical details are retained in the JSON ledger.

| Check | Result |
|---|---|
| depth-time Gram matrix is positive semidefinite | pass |
| finite-difference zeta-ratio coordinates | pass |
| time-average correlation approaches positive kernel | pass |
| principal Gamma modulus closed form | pass |
| log-frequency mean approaches polygamma target | pass |
| log-frequency variance approaches polygamma target | pass |
| signed resolvent parameter-shift equation | pass |
| positive Pade nodes remain in physical interval | pass |
| first signed one-node packet lies outside physical interval | pass |
| positive Pade model matches first eight moments | pass |
| signed Pade model matches first eight moments | pass |
| Borel and Riesz bridge | pass |
| signed finite-difference moments | pass |
| Gamma dephasing asymptotic | pass |
| depth overlap example | pass |
| finite-model long-time mean power | pass |
| character off-diagonal correlation decreases | pass |
| signed resolvent coefficient expansion | pass |

The independent test suite contains 18 additional tests. It checks the finite-difference formulas, Gram positivity, Gamma limit, hyperbolic-secant overlap, polygamma cumulants, character decoherence, resolvent transfer equation, Padé moment contract, positive versus signed node behavior, Borel–Riesz identity, figure inventory, and result ledger.

# Appendix A

# Formula registry

| Object | Formula |
|---|---|
| Signed transfer | $C_{a,\beta,\chi}(k,t) = \sum_n \mu(n)\chi(n)n^{-a-\mathrm{i}t}(1-n^{-\beta})^k$ |
| Depth-time kernel | $\mathcal{K}_{\chi,\psi}(k,\ell;\tau) = \sum_n \mu(n)^2\chi(n)\overline{\psi(n)}n^{-2a-\mathrm{i}\tau}(1-n^{-\beta})^{k+\ell}$ |
| Squarefree ratio | $D_\xi(s) = L(s,\xi)/L(2s,\xi^2)$ |
| Gamma shape | $\alpha = (2a-1)/\beta$ |
| Gamma dephasing | $\mathcal{D}_{\alpha,\beta}(\tau) = \Gamma(\alpha+\mathrm{i}\tau/\beta)/\Gamma(\alpha)$ |
| Principal modulus | $\left\lvert\mathcal{D}_{3/2,2}(\tau)\right\rvert^2 = (1+\tau^2)/\cosh(\pi\tau/2)$ |
| Depth overlap | $\operatorname{sech}^\alpha(\frac{1}{2}\log(k/\ell))$ |
| Log-frequency mean | $-\psi(\alpha)/\beta$ |
| Log-frequency variance | $\psi_1(\alpha)/\beta^2$ |
| Signed resolvent | $\mathcal{C}(z,t) = \sum_n \mu(n)\chi(n)n^{-a-\mathrm{i}t}/[1-z(1-n^{-\beta})]$ |
| Resolvent transfer | $(1-z)\mathcal{C}_a + z\mathcal{C}_{a+\beta} = L(a+\mathrm{i}t,\chi)^{-1}$ |
| Riesz companion | $\mathcal{R}(x,t) = x\sum_n \mu(n)\chi(n)n^{-a-\mathrm{i}t}\mathrm{e}^{-x/n^\beta}$ |
| Hankel determinant | $\Delta_m(t) = \det[C(i+j,t)]_{i,j=0}^{m-1}$ |
| Padé contract | $C(j,t) = \sum_{r=1}^m \lambda_r(t)\xi_r(t)^j$ for $0 \le j \le 2m-1$ |
| Principal coherence | $\Gamma_k = \lvert C_{2,2}(k,0)\rvert^2/\mathcal{K}(k,k;0)$ |

# Appendix B

# Implementation map

The complete implementation is supplied as

`projector_generated_special_functions_iv.py`.

It contains the Möbius sieve, character packets, depth-time kernel, finite time averages, exact zeta-ratio checks, Gamma asymptotics, log-frequency laws, signed resolvent, Padé packet reduction, Borel–Riesz transform, figure generation, table generation, and built-in verification ledger.

The central computational path is:

```
params = KernelParameters(a=2.0, beta=2.0, conductor=1)
data = build_number_packet_data(params, nmax=600_000)
kernel = data.positive_kernel(k + ell, tau)
signed = data.signed_transfer(k, t)
numerator, denominator = pade_from_moments(moments, order=m)
nodes, weights = pade_packet_nodes_weights(moments, order=m)
```

The positive kernel is evaluated by diagonal contraction. The signed Padé model is built from moments only. Neither path materializes a dense number-basis operator.

# Appendix C

# Complete Python source

```
from __future__ import annotations

import csv
import json
import math
from dataclasses import dataclass
from pathlib import Path
from typing import Callable, Iterable, Sequence

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch, Rectangle
import mpmath as mp
import numpy as np
from scipy import linalg
from scipy.special import digamma, gamma as gamma_fn, polygamma

mp.mp.dps = 90

ROOT = Path(__file__).resolve().parent
FIGURES = ROOT / "figures"
RESULTS = ROOT / "results"
TABLES = ROOT / "tables"
for directory in (FIGURES, RESULTS, TABLES):
    directory.mkdir(parents=True, exist_ok=True)

INK = "#17202A"
BLUE = "#2E86AB"
TEAL = "#2A9D8F"
ORANGE = "#F4A261"
MAGENTA = "#A23E8C"
RED = "#E76F51"
YELLOW = "#E9C46A"
PURPLE = "#6C5CE7"
MUTED = "#5D6D7E"
PAPER = "#FAFAF7"
GRID = "#D5D8DC"
WHITE = "#FFFFFF"
COLORS = [BLUE, TEAL, ORANGE, MAGENTA, RED, YELLOW, PURPLE]

plt.rcParams.update(
    {
        "figure.facecolor": PAPER,
        "axes.facecolor": PAPER,
        "axes.edgecolor": INK,
        "axes.labelcolor": INK,
        "xtick.color": INK,
        "ytick.color": INK,
        "text.color": INK,
```

```
        "font.size": 10.5,
        "axes.titlesize": 12.5,
        "axes.titleweight": "bold",
        "legend.frameon": False,
        "savefig.bbox": "tight",
        "savefig.facecolor": PAPER,
    }
)


def save_figure(fig: plt.Figure, stem: str) -> None:
    fig.savefig(FIGURES / f"{stem}.pdf")
    fig.savefig(FIGURES / f"{stem}.png", dpi=220)
    plt.close(fig)


def diagram_axes(figsize: tuple[float, float] = (12.6, 6.1)) -> tuple[plt.Figure, plt.Axes]:
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")
    return fig, ax


def box(
    ax: plt.Axes,
    xy: tuple[float, float],
    width: float,
    height: float,
    text: str,
    face: str = WHITE,
    edge: str = INK,
    fontsize: float = 11.0,
) -> None:
    patch = FancyBboxPatch(
        xy,
        width,
        height,
        boxstyle="round,pad=.012,rounding_size=.018",
        facecolor=face,
        edgecolor=edge,
        linewidth=1.7,
    )
    ax.add_patch(patch)
    ax.text(xy[0] + width / 2, xy[1] + height / 2, text, ha="center", va="center", fontsize=fontsize)


def arrow(
    ax: plt.Axes,
    start: tuple[float, float],
    end: tuple[float, float],
    color: str = INK,
    text: str | None = None,
    text_offset: tuple[float, float] = (0.0, 0.03),
) -> None:
    patch = FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=13, linewidth=1.6, color=color)
    ax.add_patch(patch)
    if text:
        ax.text(
            (start[0] + end[0]) / 2 + text_offset[0],
            (start[1] + end[1]) / 2 + text_offset[1],
            text,
            ha="center",
            va="center",
            fontsize=10.0,
            color=color,
        )


# ---------------------------------------------------------------------------
# Arithmetic data and packet transforms
```

```
# ---------------------------------------------------------------------------


def mobius_sieve(nmax: int) -> np.ndarray:
    mu = np.zeros(nmax + 1, dtype=np.int8)
    mu[1] = 1
    primes: list[int] = []
    composite = np.zeros(nmax + 1, dtype=bool)
    for i in range(2, nmax + 1):
        if not composite[i]:
            primes.append(i)
            mu[i] = -1
        for p in primes:
            value = i * p
            if value > nmax:
                break
            composite[value] = True
            if i % p == 0:
                mu[value] = 0
                break
            mu[value] = -mu[i]
    return mu


def prime_factors(n: int) -> list[int]:
    factors: list[int] = []
    value = n
    p = 2
    while p * p <= value:
        if value % p == 0:
            factors.append(p)
            while value % p == 0:
                value //= p
        p += 1
    if value > 1:
        factors.append(value)
    return factors


def squarefree_coprime_density(q: int) -> float:
    value = 1.0 / float(mp.zeta(2))
    for p in prime_factors(q):
        value *= p / (p + 1)
    return value


def characters_mod_five(n: np.ndarray) -> np.ndarray:
    """Return the four Dirichlet characters modulo five on an integer array."""
    n_int = np.asarray(n, dtype=np.int64)
    values = np.zeros((4, n_int.size), dtype=np.complex128)
    lookup = {1: 0, 2: 1, 4: 2, 3: 3}
    residues = n_int % 5
    for index, residue in enumerate(residues):
        if residue == 0:
            continue
        exponent = lookup[int(residue)]
        for j in range(4):
            values[j, index] = np.exp(2j * np.pi * j * exponent / 4)
    return values


@dataclass(frozen=True)
class KernelParameters:
    a: float = 2.0
    beta: float = 2.0
    conductor: int = 1

    @property
    def alpha(self) -> float:
        return (2.0 * self.a - 1.0) / self.beta

```

```
    @property
    def density(self) -> float:
        return squarefree_coprime_density(self.conductor)


@dataclass
class NumberPacketData:
    params: KernelParameters
    n: np.ndarray
    mu: np.ndarray
    y: np.ndarray
    positive_weight: np.ndarray
    signed_amplitude: np.ndarray

    def positive_kernel(self, depth: int, tau: float = 0.0) -> complex:
        logy = np.log(self.y, where=self.y > 0, out=np.full_like(self.y, -np.inf))
        depth_factor = np.exp(depth * logy) if depth else np.ones_like(self.y)
        phase = np.exp(-1j * tau * np.log(self.n))
        return complex(np.sum(self.positive_weight * depth_factor * phase))

    def signed_transfer(self, k: int, t: float = 0.0, character: np.ndarray | None = None) -> complex:
        logy = np.log(self.y, where=self.y > 0, out=np.full_like(self.y, -np.inf))
        depth_factor = np.exp(k * logy) if k else np.ones_like(self.y)
        phase = np.exp(-1j * t * np.log(self.n))
        values = self.signed_amplitude * depth_factor * phase
        if character is not None:
            values = values * character
        return complex(np.sum(values))

    def positive_companion_moment(self, k: int) -> float:
        logy = np.log(self.y, where=self.y > 0, out=np.full_like(self.y, -np.inf))
        factor = np.exp(k * logy) if k else np.ones_like(self.y)
        return float(np.sum(self.positive_weight * factor))


def build_number_packet_data(params: KernelParameters, nmax: int = 1_000_000) -> NumberPacketData:
    mu_full = mobius_sieve(nmax)
    integers = np.arange(1, nmax + 1, dtype=np.int64)
    mask = mu_full[1:] != 0
    if params.conductor > 1:
        mask &= np.gcd(integers, params.conductor) == 1
    selected = integers[mask]
    mu = mu_full[1:][mask].astype(np.float64)
    n = selected.astype(np.float64)
    y = 1.0 - n ** (-params.beta)
    positive_weight = n ** (-2.0 * params.a)
    signed_amplitude = mu * n ** (-params.a)
    return NumberPacketData(
        params=params,
        n=n,
        mu=mu,
        y=y,
        positive_weight=positive_weight,
        signed_amplitude=signed_amplitude,
    )


def exact_signed_moment(k: int, a: int = 2, beta: int = 2, dps: int = 90) -> mp.mpf:
    mp.mp.dps = dps
    return mp.fsum(
        [
            (-1) ** j * mp.binomial(k, j) / mp.zeta(a + beta * j)
            for j in range(k + 1)
        ]
    )


def exact_positive_kernel(depth: int, tau: float = 0.0, a: float = 2.0, beta: float = 2.0, dps: int = 90) -> complex:
    mp.mp.dps = dps
    total = mp.mpc(0)
    for j in range(depth + 1):
```

```
        s = 2 * a + beta * j + 1j * tau
        d = mp.zeta(s) / mp.zeta(2 * s)
        total += (-1) ** j * mp.binomial(depth, j) * d
    return complex(total)


def gamma_dephasing(alpha: float, beta: float, tau: np.ndarray | float) -> np.ndarray:
    values = np.asarray(tau, dtype=float)
    return np.array(
        [complex(mp.gamma(alpha + 1j * float(t) / beta) / mp.gamma(alpha)) for t in values.ravel()],
        dtype=np.complex128,
    ).reshape(values.shape)


def principal_modulus_squared(tau: np.ndarray | float) -> np.ndarray:
    values = np.asarray(tau, dtype=float)
    return (1.0 + values**2) / np.cosh(np.pi * values / 2.0)


def depth_overlap_limit(alpha: float, k: np.ndarray | float, ell: np.ndarray | float) -> np.ndarray:
    k_values = np.asarray(k, dtype=float)
    ell_values = np.asarray(ell, dtype=float)
    return (2.0 * np.sqrt(k_values * ell_values) / (k_values + ell_values)) ** alpha


def log_frequency_cumulants(alpha: float, beta: float, order: int) -> np.ndarray:
    values = []
    for r in range(1, order + 1):
        if r == 1:
            values.append(-float(digamma(alpha)) / beta)
        else:
            values.append(((-1.0) ** r) * float(polygamma(r - 1, alpha)) / (beta**r))
    return np.array(values)


# ---------------------------------------------------------------------------
# Time averages, resolvents, and Pade packet models
# ---------------------------------------------------------------------------


def finite_time_average(
    data: NumberPacketData,
    k: int,
    ell: int,
    tau: float,
    horizon: float,
    samples: int = 2001,
) -> complex:
    times = np.linspace(-horizon, horizon, samples)
    first = np.array([data.signed_transfer(k, float(t + tau)) for t in times])
    second = np.array([data.signed_transfer(ell, float(t)) for t in times])
    return complex(np.trapezoid(first * np.conjugate(second), times) / (2.0 * horizon))


def signed_resolvent(data: NumberPacketData, z: complex, t: float = 0.0) -> complex:
    phase = np.exp(-1j * t * np.log(data.n))
    return complex(np.sum(data.signed_amplitude * phase / (1.0 - z * data.y)))


def positive_resolvent(data: NumberPacketData, z: complex) -> complex:
    return complex(np.sum(data.positive_weight / (1.0 - z * data.y)))


def signed_resolvent_functional_residual(
    data_a: NumberPacketData,
    data_shifted: NumberPacketData,
    z: complex,
    t: float = 0.0,
) -> complex:
    left = (1.0 - z) * signed_resolvent(data_a, z, t) + z * signed_resolvent(data_shifted, z, t)
    target = complex(1.0 / mp.zeta(data_a.params.a + 1j * t))
```

```
    return left - target


def pade_from_moments(moments: Sequence[complex], order: int) -> tuple[np.ndarray, np.ndarray]:
    values = np.asarray(moments, dtype=np.complex128)
    if values.size < 2 * order:
        raise ValueError("at least 2*order moments are required")
    matrix = np.empty((order, order), dtype=np.complex128)
    rhs = np.empty(order, dtype=np.complex128)
    for row in range(order):
        n = order + row
        rhs[row] = -values[n]
        for column in range(order):
            j = column + 1
            matrix[row, column] = values[n - j]
    q_tail = np.linalg.solve(matrix, rhs)
    denominator = np.concatenate(([1.0 + 0j], q_tail))
    numerator = np.zeros(order, dtype=np.complex128)
    for n in range(order):
        total = values[n]
        for j in range(1, min(order, n) + 1):
            total += denominator[j] * values[n - j]
        numerator[n] = total
    return numerator, denominator


def pade_series_coefficients(numerator: np.ndarray, denominator: np.ndarray, count: int) -> np.ndarray:
    result = np.zeros(count, dtype=np.complex128)
    for n in range(count):
        value = numerator[n] if n < numerator.size else 0j
        for j in range(1, min(n, denominator.size - 1) + 1):
            value -= denominator[j] * result[n - j]
        result[n] = value / denominator[0]
    return result


def pade_packet_nodes_weights(moments: Sequence[complex], order: int) -> tuple[np.ndarray, np.ndarray]:
    numerator, denominator = pade_from_moments(moments, order)
    poles = np.roots(denominator[::-1])
    nodes = 1.0 / poles
    vandermonde = np.vstack([nodes**k for k in range(order)])
    weights = np.linalg.solve(vandermonde, np.asarray(moments[:order], dtype=np.complex128))
    return nodes, weights


def hankel_determinant(moments: Sequence[complex], order: int) -> complex:
    matrix = np.array([[moments[i + j] for j in range(order)] for i in range(order)], dtype=np.complex128)
    return complex(np.linalg.det(matrix))


def borel_transfer(data: NumberPacketData, x: float, t: float = 0.0) -> complex:
    phase = np.exp(-1j * t * np.log(data.n))
    return complex(np.sum(data.signed_amplitude * phase * np.exp(x * data.y)))


def riesz_companion(data: NumberPacketData, x: float, t: float = 0.0) -> complex:
    phase = np.exp(-1j * t * np.log(data.n))
    return complex(x * np.sum(data.signed_amplitude * phase * np.exp(-x / (data.n ** data.params.beta))))


# ---------------------------------------------------------------------------
# Character cross-kernels
# ---------------------------------------------------------------------------


def character_cross_kernel(
    data: NumberPacketData,
    character_a: np.ndarray,
    character_b: np.ndarray,
    depth: int,
    tau: float = 0.0,
```

```
) -> complex:
    logy = np.log(data.y, where=data.y > 0, out=np.full_like(data.y, -np.inf))
    depth_factor = np.exp(depth * logy) if depth else np.ones_like(data.y)
    phase = np.exp(-1j * tau * np.log(data.n))
    return complex(
        np.sum(
            data.positive_weight
            * character_a
            * np.conjugate(character_b)
            * depth_factor
            * phase
        )
    )


# ---------------------------------------------------------------------------
# Figures and verification
# ---------------------------------------------------------------------------


def build_figures_and_results() -> dict[str, object]:
    params = KernelParameters(a=2.0, beta=2.0, conductor=1)
    data = build_number_packet_data(params, nmax=600_000)
    visual_mask = data.n <= 12_000
    plot_data = NumberPacketData(
        params=data.params,
        n=data.n[visual_mask],
        mu=data.mu[visual_mask],
        y=data.y[visual_mask],
        positive_weight=data.positive_weight[visual_mask],
        signed_amplitude=data.signed_amplitude[visual_mask],
    )
    chars5 = characters_mod_five(data.n.astype(np.int64))

    checks: list[dict[str, object]] = []

    def record(name: str, passed: bool, detail: object) -> None:
        checks.append({"name": name, "passed": bool(passed), "detail": detail})

    # Figure 1: theory pipeline
    fig, ax = diagram_axes()
    ax.text(0.5, 0.94, "From signed arithmetic transfer to universal depth-time kernels", ha="center", fontsize=20,
     weight="bold")
    nodes = [
        (0.03, 0.58, 0.15, 0.18, "Mobius packet\n$C_k(t)$", BLUE),
        (0.22, 0.58, 0.15, 0.18, "time average\nand Gram lift", TEAL),
        (0.41, 0.58, 0.15, 0.18, "positive depth-time\nkernel $K$", ORANGE),
        (0.60, 0.58, 0.15, 0.18, "Gamma dephasing\nlimit", MAGENTA),
        (0.79, 0.58, 0.17, 0.18, "signed resolvent\nand Pade packets", RED),
    ]
    for x0, y0, w, h, text, color in nodes:
        box(ax, (x0, y0), w, h, text, face=color + "18", edge=color, fontsize=11.5)
    for left, right in zip(nodes[:-1], nodes[1:], strict=True):
        arrow(ax, (left[0] + left[2], left[1] + left[3] / 2), (right[0], right[1] + right[3] / 2))
    box(ax, (0.13, 0.18), 0.23, 0.15, "unconditional\npositive-definite theory", face=TEAL + "18", edge=TEAL,
     fontsize=12)
    box(ax, (0.41, 0.18), 0.23, 0.15, "finite signed\nmoment models", face=ORANGE + "18", edge=ORANGE, fontsize=12)
    box(ax, (0.69, 0.18), 0.20, 0.15, "RH remains a\ntime-zero coherence bound", face=RED + "18", edge=RED, fontsize
     =12)
    arrow(ax, (0.485, 0.58), (0.25, 0.33), color=TEAL)
    arrow(ax, (0.875, 0.58), (0.525, 0.33), color=ORANGE)
    arrow(ax, (0.875, 0.58), (0.79, 0.33), color=RED)
    save_figure(fig, "01_theory_pipeline")

    # Figure 2: feature map
    fig, ax = diagram_axes()
    ax.text(0.5, 0.94, "The exact feature vector behind the kernel", ha="center", fontsize=20, weight="bold")
    box(ax, (0.06, 0.55), 0.22, 0.18, "packet coordinate\n$(k,t)$", face=BLUE + "18", edge=BLUE, fontsize=14)
    box(ax, (0.39, 0.55), 0.24, 0.18, r"$v_{k,t}(n)=\sqrt{w_n}\,y_n^k n^{-it}$", face=TEAL + "18", edge=TEAL,
     fontsize=13)
```

```
    box(ax, (0.73, 0.55), 0.21, 0.18, r"$K((k,t),(\ell,s))$" + "\n" + r"$=\langle v_{k,t},v_{\ell,s}\rangle$", face=ORANGE + "18", edge=ORANGE, fontsize=12)
    arrow(ax, (0.28, 0.64), (0.39, 0.64), text="analysis")
    arrow(ax, (0.63, 0.64), (0.73, 0.64), text="Gram contraction")
    ax.text(0.5, 0.28, "Depth controls scale. Time resolves the logarithmic spectrum. The kernel keeps both coordinates without materializing a dense operator.", ha="center", fontsize=12, color=MUTED, wrap=True)
    save_figure(fig, "02_feature_map")

    # Figure 3: positive Gram matrix
    points = [(k, t) for k in (250, 1000, 4000, 16000) for t in (0.0, 1.0, 2.5)]
    gram = np.empty((len(points), len(points)), dtype=np.complex128)
    for i, (k, t) in enumerate(points):
        for j, (ell, s) in enumerate(points):
            gram[i, j] = data.positive_kernel(k + ell, t - s)
    eigenvalues = np.linalg.eigvalsh(gram.real)
    record("depth-time Gram matrix is positive semidefinite", float(np.min(eigenvalues)) > -1e-11, float(np.min(eigenvalues)))
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 5.0))
    im0 = axes[0].imshow(np.abs(gram), cmap="magma", aspect="auto")
    axes[0].set_title("absolute Gram entries")
    axes[0].set_xlabel("packet index")
    axes[0].set_ylabel("packet index")
    fig.colorbar(im0, ax=axes[0], fraction=0.046)
    axes[1].plot(np.arange(len(eigenvalues)), eigenvalues, marker="o", color=TEAL)
    axes[1].axhline(0, color=INK, lw=1)
    axes[1].set_yscale("symlog", linthresh=1e-14)
    axes[1].set_title("eigenvalues of the real Gram part")
    axes[1].set_xlabel("ordered index")
    axes[1].grid(alpha=0.18)
    fig.suptitle("The squarefree depth-time kernel is a genuine positive-definite kernel", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "03_kernel_gram")

    # Figure 4: exact finite-difference coordinates
    depths = np.arange(0, 13)
    direct = np.array([data.positive_kernel(int(m), 0.7) for m in depths])
    exact = np.array([exact_positive_kernel(int(m), 0.7, dps=100) for m in depths])
    residual = np.abs(direct - exact)
    record("finite-difference zeta-ratio coordinates", float(np.max(residual)) < 2e-9, float(np.max(residual)))
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 4.8))
    axes[0].plot(depths, direct.real, marker="o", color=BLUE, label="squarefree sum")
    axes[0].plot(depths, exact.real, marker="x", ls="--", color=ORANGE, label="finite zeta-ratio difference")
    axes[0].set_xlabel("total depth $m$")
    axes[0].set_ylabel("real part")
    axes[0].set_title("two exact coordinate systems")
    axes[0].grid(alpha=0.18)
    axes[0].legend()
    axes[1].semilogy(depths, np.maximum(residual, 1e-18), marker="o", color=RED)
    axes[1].set_xlabel("total depth $m$")
    axes[1].set_ylabel("absolute residual")
    axes[1].set_title("coordinate agreement")
    axes[1].grid(alpha=0.18)
    fig.suptitle("The kernel is a finite difference of a squarefree $L$-ratio", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "04_l_ratio_coordinates")

    # Figure 5: finite-time average convergence
    small_data = build_number_packet_data(params, nmax=2500)
    exact_corr = small_data.positive_kernel(14, 0.8)
    horizons = np.array([5, 10, 20, 40, 80, 160], dtype=float)
    approximations = np.array([finite_time_average(small_data, 6, 8, 0.8, float(T), samples=1601) for T in horizons])
    errors = np.abs(approximations - exact_corr)
    record("time-average correlation approaches positive kernel", bool(errors[-1] < errors[0]), [float(v) for v in errors])
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 4.8))
    axes[0].plot(horizons, approximations.real, marker="o", color=BLUE, label="finite average")
    axes[0].axhline(exact_corr.real, color=ORANGE, ls="--", label="exact diagonal kernel")
    axes[0].set_xlabel("time horizon $T$")
    axes[0].set_ylabel("real part")
```

```
    axes[0].set_title("ergodic diagonal extraction")
    axes[0].grid(alpha=0.18)
    axes[0].legend()
    axes[1].loglog(horizons, errors, marker="o", color=RED)
    axes[1].set_xlabel("time horizon $T$")
    axes[1].set_ylabel("absolute error")
    axes[1].set_title("finite-window convergence")
    axes[1].grid(alpha=0.18)
    fig.suptitle("The positive kernel is the long-time autocorrelation of the signed signal", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "05_time_average")

    # Figures 6-9: Gamma dephasing
    taus = np.linspace(-10, 10, 500)
    gamma_curve = gamma_dephasing(params.alpha, params.beta, taus)
    fig, ax = plt.subplots(figsize=(7.2, 6.2))
    ax.plot(gamma_curve.real, gamma_curve.imag, color=BLUE, lw=2)
    ax.scatter([1], [0], color=RED, zorder=3)
    ax.set_xlabel("real part")
    ax.set_ylabel("imaginary part")
    ax.set_title("Universal Gamma dephasing trajectory")
    ax.grid(alpha=0.18)
    ax.set_aspect("equal", adjustable="datalim")
    save_figure(fig, "06_gamma_trajectory")

    depths_for_gamma = [2000, 10000, 50000]
    tau_short = np.linspace(-5, 5, 121)
    fig, axes = plt.subplots(2, 1, figsize=(12.5, 7.0), sharex=True)
    target = gamma_dephasing(params.alpha, params.beta, tau_short)
    for depth, color in zip(depths_for_gamma, [ORANGE, TEAL, BLUE], strict=True):
        observed = np.array([
            depth ** (1j * t / params.beta) * data.positive_kernel(depth, float(t)) / data.positive_kernel(depth, 0.0)
            for t in tau_short
        ])
        axes[0].plot(tau_short, observed.real, color=color, lw=1.3, label=f"depth {depth:,}")
        axes[1].plot(tau_short, observed.imag, color=color, lw=1.3)
    axes[0].plot(tau_short, target.real, color=INK, lw=2.3, ls="--", label="Gamma limit")
    axes[1].plot(tau_short, target.imag, color=INK, lw=2.3, ls="--")
    axes[0].set_ylabel("real part")
    axes[1].set_ylabel("imaginary part")
    axes[1].set_xlabel("time lag $\\tau$")
    axes[0].set_title("dephased real component")
    axes[1].set_title("dephased imaginary component")
    for ax in axes:
        ax.grid(alpha=0.18)
    axes[0].legend(ncol=2)
    fig.suptitle("The discrete packet correlation converges to a Gamma-ratio function", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.93])
    save_figure(fig, "07_gamma_convergence")

    modulus_observed = np.abs(gamma_curve) ** 2
    modulus_closed = principal_modulus_squared(taus)
    record("principal Gamma modulus closed form", float(np.max(np.abs(modulus_observed - modulus_closed))) < 2e-13, float(np.max(np.abs(modulus_observed - modulus_closed))))
    fig, ax = plt.subplots(figsize=(12.0, 4.8))
    ax.semilogy(taus, modulus_observed, color=BLUE, lw=2, label="Gamma ratio")
    ax.semilogy(taus, modulus_closed, color=ORANGE, ls="--", lw=1.8, label=r"$(1+\tau^2)/\cosh(\pi\tau/2)$")
    ax.set_xlabel("time lag $\\tau$")
    ax.set_ylabel("squared modulus")
    ax.set_title(r"Closed dephasing envelope for $a=\beta=2$")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "08_principal_modulus")

    phase = np.unwrap(np.angle(gamma_curve))
    phase_derivative = np.gradient(phase, taus)
    target_phase_derivative = np.real(np.array([complex(mp.digamma(params.alpha + 1j * t / params.beta)) for t in taus])) / params.beta
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 4.8))
```

```
    axes[0].plot(taus, phase, color=MAGENTA)
    axes[0].set_xlabel("time lag $\\tau$")
    axes[0].set_ylabel("phase")
    axes[0].set_title("unwrapped packet phase")
    axes[0].grid(alpha=0.18)
    axes[1].plot(taus, phase_derivative, color=BLUE, label="numerical derivative")
    axes[1].plot(taus, target_phase_derivative, color=ORANGE, ls="--", label="digamma law")
    axes[1].set_xlabel("time lag $\\tau$")
    axes[1].set_ylabel("phase slope")
    axes[1].set_title("exact group-delay law")
    axes[1].grid(alpha=0.18)
    axes[1].legend()
    fig.suptitle("The limiting phase is controlled by the digamma function", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "09_phase_digamma")

    # Figures 10-11: log-frequency law and cumulants
    depth_log = 100000
    logy = np.log(data.y, where=data.y > 0, out=np.full_like(data.y, -np.inf))
    probability = data.positive_weight * np.exp(depth_log * logy)
    probability /= np.sum(probability)
    centered_log = np.log(data.n) - math.log(depth_log) / params.beta
    sample_mean = float(np.sum(probability * centered_log))
    sample_variance = float(np.sum(probability * (centered_log - sample_mean) ** 2))
    target_cumulants = log_frequency_cumulants(params.alpha, params.beta, 4)
    record("log-frequency mean approaches polygamma target", abs(sample_mean - target_cumulants[0]) < 0.01, [
     sample_mean, float(target_cumulants[0])])
    record("log-frequency variance approaches polygamma target", abs(sample_variance - target_cumulants[1]) < 0.02,
     [sample_variance, float(target_cumulants[1])])
    fig, ax = plt.subplots(figsize=(12.0, 5.0))
    bins = np.linspace(-2.0, 2.0, 90)
    hist, edges = np.histogram(centered_log, bins=bins, weights=probability, density=True)
    centers = (edges[:-1] + edges[1:]) / 2
    ax.plot(centers, hist, drawstyle="steps-mid", color=BLUE, lw=1.8, label="discrete tilted packet")
    u = np.logspace(-4, 4, 200000)
    z = -np.log(u) / params.beta
    density_u = u ** (params.alpha - 1) * np.exp(-u) / gamma_fn(params.alpha)
    density_z = params.beta * u * density_u
    order = np.argsort(z)
    ax.plot(z[order], density_z[order], color=ORANGE, ls="--", lw=2, label="log-Gamma limit")
    ax.set_xlim(-2, 2)
    ax.set_xlabel(r"$\log n-\beta^{-1}\log m$")
    ax.set_ylabel("density")
    ax.set_title("Universal centered log-frequency profile")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "10_log_frequency_profile")

    depth_values = np.array([1000, 3000, 10000, 30000, 100000], dtype=int)
    means = []
    variances = []
    for depth in depth_values:
        p = data.positive_weight * np.exp(depth * logy)
        p /= np.sum(p)
        zvals = np.log(data.n) - math.log(depth) / params.beta
        mean = float(np.sum(p * zvals))
        variance = float(np.sum(p * (zvals - mean) ** 2))
        means.append(mean)
        variances.append(variance)
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 4.8))
    axes[0].semilogx(depth_values, means, marker="o", color=BLUE)
    axes[0].axhline(target_cumulants[0], color=ORANGE, ls="--", label="polygamma limit")
    axes[0].set_xlabel("depth $m$")
    axes[0].set_ylabel("centered mean")
    axes[0].set_title("mean log-frequency")
    axes[0].grid(alpha=0.18)
    axes[0].legend()
    axes[1].semilogx(depth_values, variances, marker="o", color=TEAL)
    axes[1].axhline(target_cumulants[1], color=ORANGE, ls="--", label="polygamma limit")
    axes[1].set_xlabel("depth $m$")
    axes[1].set_ylabel("variance")
```

```
    axes[1].set_title("log-frequency variance")
    axes[1].grid(alpha=0.18)
    axes[1].legend()
    fig.suptitle("Logarithmic spectral cumulants converge to explicit polygamma constants", fontsize=17, weight="
     bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "11_log_cumulants")

    # Figures 12-14: depth overlap
    ratios = np.logspace(-2, 2, 300)
    limit_overlap = depth_overlap_limit(params.alpha, ratios, np.ones_like(ratios))
    fig, ax = plt.subplots(figsize=(12.0, 4.8))
    ax.semilogx(ratios, limit_overlap, color=BLUE, lw=2)
    ax.axvline(1, color=INK, lw=1)
    ax.set_xlabel("depth ratio $k/\\ell$")
    ax.set_ylabel("normalized overlap")
    ax.set_title(r"Universal scale overlap $\operatorname{sech}^{\alpha}(\frac{1}{2}\log(k/\ell))$")
    ax.grid(alpha=0.18)
    save_figure(fig, "12_depth_overlap_limit")

    base_depths = [2000, 10000, 50000]
    ratio_samples = np.array([0.25, 0.5, 1.0, 2.0, 4.0])
    fig, ax = plt.subplots(figsize=(12.0, 5.0))
    for base, color in zip(base_depths, [ORANGE, TEAL, BLUE], strict=True):
        values = []
        for ratio in ratio_samples:
            k = int(round(base * ratio))
            ell = base
            observed = data.positive_kernel(k + ell, 0.0).real / math.sqrt(
                data.positive_kernel(2 * k, 0.0).real * data.positive_kernel(2 * ell, 0.0).real
            )
            values.append(observed)
        ax.plot(ratio_samples, values, marker="o", color=color, label=f"base {base:,}")
    ax.plot(ratio_samples, depth_overlap_limit(params.alpha, ratio_samples, 1.0), color=INK, ls="--", lw=2, label="
     limit")
    ax.set_xscale("log")
    ax.set_xlabel("depth ratio $k/\\ell$")
    ax.set_ylabel("normalized overlap")
    ax.set_title("Finite packet overlaps approach the hyperbolic-secant law")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "13_depth_overlap_convergence")

    k_grid = np.logspace(2.5, 5.0, 55)
    overlap_grid = np.empty((k_grid.size, k_grid.size))
    for i, k in enumerate(k_grid):
        for j, ell in enumerate(k_grid):
            overlap_grid[i, j] = depth_overlap_limit(params.alpha, k, ell)
    fig, ax = plt.subplots(figsize=(7.0, 6.0))
    im = ax.imshow(overlap_grid, origin="lower", cmap="viridis", extent=[2.5, 5.0, 2.5, 5.0], aspect="auto", vmin=0,
      vmax=1)
    ax.set_xlabel(r"$\log_{10}\ell$")
    ax.set_ylabel(r"$\log_{10}k$")
    ax.set_title("Universal overlap geometry on log depth")
    fig.colorbar(im, ax=ax, fraction=0.046, label="overlap")
    save_figure(fig, "14_depth_overlap_heatmap")

    # Figures 15-16: character cross-kernels
    char_depths = [100, 1000, 10000, 100000]
    char_corr_matrices = []
    for depth in char_depths:
        matrix = np.empty((4, 4), dtype=np.complex128)
        for i in range(4):
            for j in range(4):
                numerator = character_cross_kernel(data, chars5[i], chars5[j], depth, 0.0)
                denominator = math.sqrt(
                    abs(character_cross_kernel(data, chars5[i], chars5[i], depth, 0.0))
                    * abs(character_cross_kernel(data, chars5[j], chars5[j], depth, 0.0))
                )
                matrix[i, j] = numerator / denominator
        char_corr_matrices.append(matrix)
```

```
    fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.8))
    for ax, matrix, depth in zip(axes, char_corr_matrices, char_depths, strict=True):
        image = ax.imshow(np.abs(matrix), cmap="magma", vmin=0, vmax=1)
        ax.set_title(f"depth {depth:,}", fontsize=10.5)
        ax.set_xticks(range(4))
        ax.set_yticks(range(4))
    fig.colorbar(image, ax=axes.ravel().tolist(), fraction=0.018, pad=0.02, label="normalized magnitude")
    fig.suptitle("Distinct modulo-five character packets decohere with depth", fontsize=17, weight="bold")
    fig.subplots_adjust(left=0.05, right=0.92, bottom=0.12, top=0.82, wspace=0.18)
    save_figure(fig, "15_character_gram")

    max_offdiag = []
    for matrix in char_corr_matrices:
        mask = ~np.eye(4, dtype=bool)
        max_offdiag.append(float(np.max(np.abs(matrix[mask]))))
    fig, ax = plt.subplots(figsize=(11.5, 4.8))
    ax.loglog(char_depths, max_offdiag, marker="o", color=BLUE, label="largest off-diagonal")
    reference = max_offdiag[0] * (np.array(char_depths) / char_depths[0]) ** (-1 / (2 * params.beta))
    ax.loglog(char_depths, reference, color=ORANGE, ls="--", label=r"$m^{-1/(2\beta)}$")
    ax.set_xlabel("total depth $m$")
    ax.set_ylabel("largest normalized cross-correlation")
    ax.set_title("Depth-time character decoherence retains the same unconditional exponent")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "16_character_decoherence")

    # Figures 17-19: signed Mobius process
    time_axis = np.linspace(-25, 25, 801)
    signed_depths = [20, 100, 500]
    phase_matrix = np.exp(-1j * np.outer(time_axis, np.log(plot_data.n)))
    plot_logy = np.log(plot_data.y, where=plot_data.y > 0, out=np.full_like(plot_data.y, -np.inf))
    signed_series: dict[int, np.ndarray] = {}
    fig, axes = plt.subplots(len(signed_depths), 1, figsize=(12.5, 7.6), sharex=True)
    for ax, depth, color in zip(axes, signed_depths, [ORANGE, TEAL, BLUE], strict=True):
        coefficients = plot_data.signed_amplitude * np.exp(depth * plot_logy)
        values = phase_matrix @ coefficients
        signed_series[depth] = values
        ax.plot(time_axis, values.real, color=color, lw=1.2, label="real")
        ax.plot(time_axis, values.imag, color=MAGENTA, lw=1.0, alpha=0.8, label="imaginary")
        ax.set_ylabel(f"$k={depth}$")
        ax.grid(alpha=0.14)
    axes[0].legend(ncol=2)
    axes[-1].set_xlabel("logarithmic time $t$")
    fig.suptitle("The signed Mobius transfer is a deterministic quasiperiodic signal", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.93])
    save_figure(fig, "17_signed_signal")

    k_process = 300
    coefficients = plot_data.signed_amplitude * np.exp(k_process * plot_logy)
    values = phase_matrix @ coefficients
    energy = plot_data.positive_kernel(2 * k_process, 0.0).real
    normalized = values / math.sqrt(energy)
    fig, axes = plt.subplots(2, 1, figsize=(12.5, 6.5), sharex=True)
    axes[0].plot(time_axis, normalized.real, color=BLUE, label="real")
    axes[0].plot(time_axis, normalized.imag, color=ORANGE, label="imaginary")
    axes[0].set_ylabel("normalized amplitude")
    axes[0].set_title("unit-average-power normalization")
    axes[0].grid(alpha=0.18)
    axes[0].legend()
    axes[1].plot(time_axis, np.abs(normalized) ** 2, color=TEAL)
    axes[1].axhline(1, color=INK, ls="--", label="long-time mean power")
    axes[1].set_xlabel("logarithmic time $t$")
    axes[1].set_ylabel("instantaneous power")
    axes[1].set_title("time zero is one distinguished recurrence time")
    axes[1].grid(alpha=0.18)
    axes[1].legend()
    fig.tight_layout()
    save_figure(fig, "18_normalized_process")

    coherence_ks = np.unique(np.logspace(1, 4.3, 90).astype(int))
    coherences = []
```

```
    for k in coherence_ks:
        signed = data.signed_transfer(int(k), 0.0)
        positive = data.positive_kernel(2 * int(k), 0.0).real
        coherences.append(abs(signed) ** 2 / positive)
    fig, ax = plt.subplots(figsize=(12.0, 4.8))
    ax.loglog(coherence_ks, coherences, color=BLUE, lw=1.4)
    ax.set_xlabel("depth $k$")
    ax.set_ylabel(r"$\Gamma_k=|C_k(0)|^2/E_k$")
    ax.set_title("Finite principal Mobius coherence data")
    ax.grid(alpha=0.18)
    ax.text(0.02, 0.06, "Finite truncation and finite depth only. No asymptotic conclusion is drawn.", transform=ax.transAxes, color=MUTED, fontsize=9.5)
    save_figure(fig, "19_time_zero_coherence")

    # Figures 20-21: signed resolvent and functional equation
    grid_x = np.linspace(-2.2, 0.95, 110)
    grid_y = np.linspace(-2.0, 2.0, 100)
    magnitude = np.empty((grid_y.size, grid_x.size))
    for row, imag in enumerate(grid_y):
        zrow = grid_x + 1j * imag
        denominators = 1.0 - zrow[:, None] * plot_data.y[None, :]
        values = np.sum(plot_data.signed_amplitude[None, :] / denominators, axis=1)
        magnitude[row, :] = np.abs(values)
    fig, ax = plt.subplots(figsize=(8.0, 6.0))
    image = ax.imshow(np.log10(np.maximum(magnitude, 1e-12)), origin="lower", extent=[grid_x[0], grid_x[-1], grid_y[0], grid_y[-1]], cmap="magma", aspect="auto")
    ax.set_xlabel("real $z$")
    ax.set_ylabel("imaginary $z$")
    ax.set_title("Signed Mobius resolvent on the slit-plane side of the cut")
    fig.colorbar(image, ax=ax, fraction=0.046, label=r"$\log_{10}|C(z)|$")
    save_figure(fig, "20_signed_resolvent")

    shifted_data = build_number_packet_data(KernelParameters(a=4.0, beta=2.0, conductor=1), nmax=600_000)
    z_points = [complex(-0.8, 0.2), complex(0.25, 0.4), complex(0.7, -0.3)]
    functional_residuals = [abs(signed_resolvent_functional_residual(data, shifted_data, z)) for z in z_points]
    record("signed resolvent parameter-shift equation", max(functional_residuals) < 5e-7, functional_residuals)
    fig, ax = plt.subplots(figsize=(11.5, 4.8))
    labels = [f"{z.real:+.2f}{z.imag:+.2f}i" for z in z_points]
    ax.bar(np.arange(len(labels)), functional_residuals, color=[BLUE, TEAL, ORANGE])
    ax.set_yscale("log")
    ax.set_xticks(np.arange(len(labels)), labels)
    ax.set_ylabel("absolute residual")
    ax.set_title("Numerical certificate for the exact resolvent transfer equation")
    ax.grid(axis="y", alpha=0.18)
    save_figure(fig, "21_resolvent_functional_equation")

    # Figures 22-24: positive and signed Pade packet models
    signed_moments = np.array([complex(exact_signed_moment(k)) for k in range(16)])
    positive_moments = np.array([data.positive_companion_moment(k) for k in range(16)], dtype=np.complex128)
    positive_nodes, positive_weights = pade_packet_nodes_weights(positive_moments, 4)
    signed_nodes, signed_weights = pade_packet_nodes_weights(signed_moments, 4)
    record("positive Pade nodes remain in physical interval", bool(np.all(np.abs(positive_nodes.imag) < 1e-8) and np.all((positive_nodes.real >= -1e-8) & (positive_nodes.real < 1 + 1e-8))), [complex(v) for v in positive_nodes])
    record("first signed one-node packet lies outside physical interval", abs((signed_moments[1] / signed_moments[0]).real + 0.51981775) < 1e-6, complex(signed_moments[1] / signed_moments[0]))
    fig, axes = plt.subplots(1, 2, figsize=(12.6, 4.5))
    axes[0].scatter(positive_nodes.real, positive_nodes.imag, s=80, color=TEAL)
    axes[0].axvspan(0, 1, color=TEAL, alpha=0.08)
    axes[0].axhline(0, color=GRID)
    axes[0].set_xlim(-0.1, 1.1)
    axes[0].set_ylim(-0.1, 0.1)
    axes[0].set_title("positive companion nodes")
    axes[0].set_xlabel("real part")
    axes[0].set_ylabel("imaginary part")
    axes[1].scatter(signed_nodes.real, signed_nodes.imag, s=80, color=RED)
    axes[1].axvspan(0, 1, color=TEAL, alpha=0.08)
    axes[1].axhline(0, color=GRID)
    axes[1].set_xlim(min(-0.1, signed_nodes.real.min() - 0.05), 1.1)
    axes[1].set_ylim(-0.1, 0.1)
    axes[1].set_title("signed Mobius packet nodes")
    axes[1].set_xlabel("real part")
```

```
    fig.suptitle("Positive reduction stays on the physical spectrum. Signed reduction need not.", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "22_pade_nodes")

    positive_num, positive_den = pade_from_moments(positive_moments, 4)
    signed_num, signed_den = pade_from_moments(signed_moments, 4)
    positive_reconstructed = pade_series_coefficients(positive_num, positive_den, 12)
    signed_reconstructed = pade_series_coefficients(signed_num, signed_den, 12)
    positive_errors = np.abs(positive_reconstructed - positive_moments[:12])
    signed_errors = np.abs(signed_reconstructed - signed_moments[:12])
    record("positive Pade model matches first eight moments", float(np.max(positive_errors[:8])) < 1e-12, float(np.max(positive_errors[:8])))
    record("signed Pade model matches first eight moments", float(np.max(signed_errors[:8])) < 1e-11, float(np.max(signed_errors[:8])))
    fig, ax = plt.subplots(figsize=(11.5, 4.8))
    ax.semilogy(range(12), np.maximum(positive_errors, 1e-18), marker="o", color=TEAL, label="positive companion")
    ax.semilogy(range(12), np.maximum(signed_errors, 1e-18), marker="o", color=RED, label="signed Mobius")
    ax.axvline(7.5, color=INK, ls="--", label="exact contract ends")
    ax.set_xlabel("moment order")
    ax.set_ylabel("absolute error")
    ax.set_title("Four-packet Pade models match the first eight moments exactly")
    ax.grid(alpha=0.18)
    ax.legend()
    save_figure(fig, "23_pade_moment_matching")

    orders = np.arange(1, 8)
    positive_determinants = np.array([hankel_determinant(positive_moments, int(order)).real for order in orders])
    signed_determinants = np.array([hankel_determinant(signed_moments, int(order)).real for order in orders])
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 4.8))
    axes[0].semilogy(orders, positive_determinants, marker="o", color=TEAL)
    axes[0].set_xlabel("Hankel order")
    axes[0].set_ylabel("determinant")
    axes[0].set_title("positive determinants")
    axes[0].grid(alpha=0.18)
    axes[1].plot(orders, np.sign(signed_determinants) * np.log10(np.maximum(np.abs(signed_determinants), 1e-300)), marker="o", color=RED)
    axes[1].axhline(0, color=GRID)
    axes[1].set_xlabel("Hankel order")
    axes[1].set_ylabel(r"signed $\log_{10}|\Delta_m|$")
    axes[1].set_title("signed determinants change sign")
    axes[1].grid(alpha=0.18)
    fig.suptitle("The signed moment problem is quasi-definite rather than positive", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "24_hankel_contrast")

    # Figure 25: Borel/Riesz bridge
    x_values = np.linspace(0.2, 12, 150)
    borel_values = np.array([borel_transfer(plot_data, float(x)) for x in x_values])
    riesz_values = np.array([riesz_companion(plot_data, float(x)) for x in x_values])
    bridge_residual = np.max(np.abs(borel_values - np.exp(x_values) * riesz_values / x_values))
    record("Borel and Riesz bridge", float(bridge_residual) < 2e-10, float(bridge_residual))
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 4.8))
    axes[0].plot(x_values, borel_values.real, color=BLUE, label="Borel transform")
    axes[0].plot(x_values, (np.exp(x_values) * riesz_values / x_values).real, color=ORANGE, ls="--", label="Riesz form")
    axes[0].set_xlabel("$x$")
    axes[0].set_ylabel("real value")
    axes[0].set_title("two exact continuous coordinates")
    axes[0].grid(alpha=0.18)
    axes[0].legend()
    axes[1].semilogy(x_values, np.maximum(np.abs(borel_values - np.exp(x_values) * riesz_values / x_values), 1e-18), color=RED)
    axes[1].set_xlabel("$x$")
    axes[1].set_ylabel("absolute residual")
    axes[1].set_title("bridge residual")
    axes[1].grid(alpha=0.18)
    fig.suptitle("The signed depth sequence and the Riesz function share one Borel transform", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "25_borel_riesz_bridge")
```

```
    # Figure 26: quantum operator diagram
    fig, ax = diagram_axes()
    ax.text(0.5, 0.94, "Number-basis realization of the signed recurrence process", ha="center", fontsize=20, weight="bold")
    box(ax, (0.04, 0.58), 0.18, 0.16, r"$|\phi_a\rangle$" + "\nnumber-basis seed", face=BLUE + "18", edge=BLUE, fontsize=13)
    box(ax, (0.29, 0.58), 0.18, 0.16, r"$e^{-kK_\beta}$" + "\nscale filter", face=TEAL + "18", edge=TEAL, fontsize=13)
    box(ax, (0.54, 0.58), 0.18, 0.16, "$e^{-itH}$\nlogarithmic phase", face=ORANGE + "18", edge=ORANGE, fontsize=13)
    box(ax, (0.79, 0.58), 0.17, 0.16, r"$M_\chi$" + "\narithmetic sign", face=MAGENTA + "18", edge=MAGENTA, fontsize=13)
    for left, right in [((0.22, 0.66), (0.29, 0.66)), ((0.47, 0.66), (0.54, 0.66)), ((0.72, 0.66), (0.79, 0.66))]:
        arrow(ax, left, right)
    box(ax, (0.13, 0.20), 0.27, 0.15, "signed amplitude\n$C_k(t)$", face=RED + "18", edge=RED, fontsize=14)
    box(ax, (0.60, 0.20), 0.27, 0.15, "time-averaged power\n" + r"$K_{2k}(\tau)$", face=TEAL + "18", edge=TEAL, fontsize=14)
    arrow(ax, (0.88, 0.58), (0.27, 0.35), color=RED, text="coherent readout", text_offset=(0, -0.04))
    arrow(ax, (0.88, 0.58), (0.73, 0.35), color=TEAL, text="autocorrelation", text_offset=(0, 0.04))
    save_figure(fig, "26_quantum_operator")

    # Figure 27: status boundary
    fig, ax = diagram_axes()
    ax.text(0.5, 0.94, "What is proved and what remains open", ha="center", fontsize=21, weight="bold")
    box(ax, (0.06, 0.56), 0.25, 0.22, "proved\npositive kernel\nGamma dephasing\ncharacter decoherence", face=TEAL + "20", edge=TEAL, fontsize=12.5)
    box(ax, (0.375, 0.56), 0.25, 0.22, "proved conditionally\nsigned Pade packets\nwhen Hankel systems\ndo not break down", face=ORANGE + "20", edge=ORANGE, fontsize=12.5)
    box(ax, (0.69, 0.56), 0.25, 0.22, "open\nsubpolynomial time-zero\nMobius coherence", face=RED + "20", edge=RED, fontsize=13)
    ax.text(0.5, 0.28, "The positive autocorrelation is universal. RH remains the claim that one distinguished recurrence time is not polynomially exceptional.", ha="center", fontsize=12.5, color=MUTED, wrap=True)
    save_figure(fig, "27_status_boundary")

    # Figure 28: family map
    fig, ax = diagram_axes()
    ax.text(0.5, 0.95, "Function family generated in Paper IV", ha="center", fontsize=21, weight="bold")
    items = [
        (0.06, 0.66, "depth-time kernel\n" + r"$K_{k,\ell}(\tau)$", BLUE),
        (0.38, 0.66, "Gamma dephasing\n" + r"$\Gamma(\alpha+i\tau/\beta)/\Gamma(\alpha)$", TEAL),
        (0.70, 0.66, "Mobius resolvent\n" + r"$C(z,t)$", ORANGE),
        (0.06, 0.27, "log-frequency law\npolygamma cumulants", PURPLE),
        (0.38, 0.27, "character cross-kernels\ndecoherence", MAGENTA),
        (0.70, 0.27, "signed Pade packets\nfinite moment contracts", RED),
    ]
    for x0, y0, text, color in items:
        box(ax, (x0, y0), 0.24, 0.17, text, face=color + "18", edge=color, fontsize=11.5)
    arrow(ax, (0.30, 0.745), (0.38, 0.745))
    arrow(ax, (0.62, 0.745), (0.70, 0.745))
    arrow(ax, (0.18, 0.66), (0.18, 0.44))
    arrow(ax, (0.50, 0.66), (0.50, 0.44))
    arrow(ax, (0.82, 0.66), (0.82, 0.44))
    save_figure(fig, "28_function_family_map")

    # Additional built-in checks
    # Exact low-order signed moments against direct truncation
    signed_direct = np.array([data.signed_transfer(k, 0.0).real for k in range(7)])
    signed_exact = np.array([float(exact_signed_moment(k)) for k in range(7)])
    record("signed finite-difference moments", float(np.max(np.abs(signed_direct - signed_exact))) < 2e-6, float(np.max(np.abs(signed_direct - signed_exact))))

    # Gamma asymptotic at several lags
    gamma_errors = []
    depth_check = 50000
    for tau in (0.0, 0.7, 1.5, 3.0):
        observed = depth_check ** (1j * tau / params.beta) * data.positive_kernel(depth_check, tau) / data.positive_kernel(depth_check, 0.0)
        target = complex(mp.gamma(params.alpha + 1j * tau / params.beta) / mp.gamma(params.alpha))
        gamma_errors.append(abs(observed - target))
    record("Gamma dephasing asymptotic", max(gamma_errors) < 0.03, gamma_errors)
```

```
    # Depth overlap example k=4 ell=1 after scaling
    expected_overlap = 0.8 ** params.alpha
    observed_overlap = data.positive_kernel(50000, 0.0).real / math.sqrt(
        data.positive_kernel(80000, 0.0).real * data.positive_kernel(20000, 0.0).real
    )
    record("depth overlap example", abs(observed_overlap - expected_overlap) < 0.02, [observed_overlap,
     expected_overlap])

    # Long-time mean power in a finite model
    finite_average = finite_time_average(small_data, 4, 4, 0.0, 200.0, samples=3001)
    exact_power = small_data.positive_kernel(8, 0.0)
    record("finite-model long-time mean power", abs(finite_average - exact_power) < 0.03, [finite_average,
     exact_power])

    # Character off-diagonal decreases between first and last depth
    record("character off-diagonal correlation decreases", max_offdiag[-1] < max_offdiag[0], max_offdiag)

    # Signed resolvent coefficients match moment sequence
    z_test = 0.12
    series_approx = sum(signed_moments[k] * z_test**k for k in range(14))
    direct_resolvent = signed_resolvent(data, z_test)
    record("signed resolvent coefficient expansion", abs(series_approx - direct_resolvent) < 2e-7, abs(series_approx
      - direct_resolvent))

    # Save tables
    gamma_table_path = TABLES / "gamma_dephasing.csv"
    with gamma_table_path.open("w", newline="") as handle:
        writer = csv.writer(handle)
        writer.writerow(["tau", "real", "imag", "modulus_squared"])
        for tau in (-4, -2, -1, 0, 1, 2, 4):
            value = complex(mp.gamma(params.alpha + 1j * tau / params.beta) / mp.gamma(params.alpha))
            writer.writerow([tau, value.real, value.imag, abs(value) ** 2])

    pade_table_path = TABLES / "pade_nodes.csv"
    with pade_table_path.open("w", newline="") as handle:
        writer = csv.writer(handle)
        writer.writerow(["family", "index", "node_real", "node_imag", "weight_real", "weight_imag"])
        for family, nodes, weights in [
            ("positive", positive_nodes, positive_weights),
            ("signed", signed_nodes, signed_weights),
        ]:
            order = np.argsort(nodes.real)
            for index, selected in enumerate(order):
                writer.writerow(
                    [
                        family,
                        index,
                        nodes[selected].real,
                        nodes[selected].imag,
                        weights[selected].real,
                        weights[selected].imag,
                    ]
                )

    coherence_table_path = TABLES / "coherence.csv"
    with coherence_table_path.open("w", newline="") as handle:
        writer = csv.writer(handle)
        writer.writerow(["k", "gamma_k"])
        for k, value in zip(coherence_ks, coherences, strict=True):
            writer.writerow([int(k), float(value)])

    passed = sum(int(item["passed"]) for item in checks)
    result = {
        "title": "Projector-Generated Special Functions IV",
        "author": "Komorebi P.",
        "checks": len(checks),
        "passed": passed,
        "figure_pairs": 28,
        "parameters": {
            "a": params.a,
            "beta": params.beta,
```

```
            "alpha": params.alpha,
            "nmax": int(data.n[-1]),
        },
        "principal_closed_form_residual": float(np.max(np.abs(modulus_observed - modulus_closed))),
        "gamma_asymptotic_errors": [float(v) for v in gamma_errors],
        "signed_one_node": complex(signed_moments[1] / signed_moments[0]).real,
        "checks_detail": checks,
    }
    (RESULTS / "results.json").write_text(json.dumps(result, indent=2, default=str) + "\n")
    with (RESULTS / "checks.csv").open("w", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=["name", "passed", "detail"])
        writer.writeheader()
        for item in checks:
            writer.writerow({"name": item["name"], "passed": item["passed"], "detail": item["detail"]})
    return result


def main() -> None:
    result = build_figures_and_results()
    print(json.dumps(result, indent=2, default=str))
    if result["passed"] != result["checks"]:
        raise SystemExit("one or more built-in checks failed")


if __name__ == "__main__":
    main()
```

## C.1 Reproduction

From the source directory, run

```
python projector_generated_special_functions_iv.py
python generate_tables.py
python -m pytest -q
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_IV.tex
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_IV.tex
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_IV.tex
```

## C.2 Source boundary

The external historical inputs, prior special-function literature, and the 2026 logarithmic-spectrum quantum construction belong in the consolidated end-of-series bibliography. No historical priority claim is made for every displayed specialization. The paper claims the formulas and theorems proved here, the generated function organization, and the computational certificates supplied with the release.